Autonomous Rover for Warehouse Rack Inventory Management

Date: December 6, 2025

Contents

1 Executive Summary **4**
- 1.1 Problem Statement . . . 4
- 1.2 Solution Summary . . . 4
- 1.3 Key Metrics and Innovation Highlights . . . 5

2 Problem Understanding & Motivation **6**

3 System Architecture **7**
- 3.1 Block diagram (Hardware & Software) . . . 7
- 3.2 Hardware subsystems . . . 7
  - 3.2.1 Navigation and Positioning . . . 7
  - 3.2.2 Scanning . . . 9
  - 3.2.3 Power . . . 9
- 3.3 Software Stack . . . 10
  - 3.3.1 SLAM . . . 10
  - 3.3.2 QR . . . 11
  - 3.3.3 HMI . . . 11
- 3.4 Data flow diagram . . . 14

4 Hardware Design **15**
- 4.1 Chassis Specifications . . . 15
- 4.2 Drive System . . . 15
- 4.3 Vertical Lift Mechanism . . . 16
- 4.4 Sensor Justification . . . 16
- 4.5 Power Systems . . . 17
- 4.6 Safety Features . . . 18

5 Software Architecture **19**
- 5.1 Navigation Pipeline . . . 19
  - 5.1.1 SLAM . . . 19
  - 5.1.2 Planning . . . 20
  - 5.1.3 Avoidance . . . 21
- 5.2 Vision Pipeline . . . 22
  - 5.2.1 High-Level System Architecture . . . 22
  - 5.2.2 Data Flow Overview . . . 24
  - 5.2.3 Design choices and their justifications . . . 25
  - 5.2.4 Control Flow: System Initialization Sequence . . . 25

5.2.5 Scanner Subsystem: Optimized Frame Acquisition . . . 26
5.2.6 QR Detection Pipeline . . . 27
5.2.7 Processing and Data Logging Pipeline . . . 29
5.3 HMI Design . . . 29
5.3.1 Front-End Structure . . . 30
5.3.2 Back-End Architecture . . . 30
5.3.3 Role in the Overall System . . . 31
5.4 Communication Protocols . . . 31
5.4.1 Software Stack Overview . . . 31
5.4.2 Module-wise Architecture . . . 32

6 Integration & Control **33**
6.1 Hardware & Software Interactions . . . 33
6.2 Control Flow . . . 33
6.3 Fail-safe Mechanisms . . . 34

7 Testing & Validation Summary **35**
7.1 Odometry Validation . . . 35
7.2 Z Motion Feedback . . . 35
7.3 Camera and Vision Pipeline . . . 35

8 Innovation & Novel Contributions **37**
8.1 Design Choices . . . 37
8.1.1 Software Architecture . . . 37
8.1.2 Geometric Projection before further Processing . . . 37
8.1.3 Duplicate Suppression in Physical Space . . . 38
8.1.4 Asynchronous Orchestration and Fault Tolerance . . . 38
8.2 Algorithm Optimisations . . . 39
8.2.1 Contrast Aware QR Preprocessing . . . 39
8.2.2 Detector and Decoder Pairing . . . 39
8.3 Cost and Efficiency Innovations . . . 39

9 Limitations & Future Scope **40**
9.1 Known Limitations . . . 40
9.2 Commercialization Path . . . 40
9.3 Scalability . . . 40

10 Innovation Highlights **41**
10.1 Innovative Approaches . . . 41
10.2 Algorithms Used . . . 41
10.3 Status Indicators and Alerts . . . 42

11 Testing & Validation **43**
11.1 Z-axis Motion . . . 43
11.2 Drive Motion . . . 43

11.3 Image capture . . . 43
11.4 Navigation . . . 44

12 Maintenance **46**
12.1 Preventive Maintenance . . . 46
12.2 Replacement Parts . . . 46
12.2.1 Electrical and Control Units . . . 46
12.2.2 Mechanical Structures . . . 47
12.3 Storage and Transport . . . 47

13 References & Appendix **48**

List of Figures

2.1 Overview of key operational stages in warehouse logistics. Source: https://anchanto.com/wms-robotics-for-efficient-warehouse-operations/ . . . 6

3.1 High level interaction between the rover's hardware subsystems and software stack. . 7
3.2 Block diagram of the robot's navigation and control hardware architecture. . . . 7
3.3 Power distribution diagram for battery, buck converters, and controllers. . . . 9
3.4 Software pipeline from sensors and vision node to SLAM, Nav2, and motor control. . 10
3.5 Complete Vision Pipeline from raw image to data logging . . . 11
3.6 Block diagram of the Human Machine Interface . . . 11
3.7 Interface of website . . . 12
3.8 Block diagram of the Data flow from the sensors to the HMI . . . 14

5.1 Mapping Using SLAM . . . 22
5.2 High level architecture of Vision Pipeline . . . 23
5.3 We use the above three coordinate frames to establish a 3D projection of image to a plane parallel to the rack (Geometric Rectification) . . . 27
5.4 QR code processing pipeline from projected image to inventory records. . . . 28
5.5 Real-time warehouse inventory dashboard showing rack status, search, and event logs 30

11.1 QR code processing pipeline from projected image to inventory records. . . . 43
11.2 QR code processing pipeline from projected image to inventory records. . . . 43
11.3 Inventory.csv . . . 44
11.4 Inventory.csv . . . 44
11.5 Map generation via Nav2 and SLAM . . . 45

Chapter 1

Executive Summary

1.1 Problem Statement

Modern high output warehouses operate with tightly coupled Warehouse Management Systems and fast pick and pack workflows, yet the ground truth on the racks often drifts away from what the database reports. Inventory updates are typically driven by manual cycle counts and operator scans, which introduce **latency**, **human error** and **inconsistent** coverage, particularly for higher rack levels and densely packed locations. This mismatch between the inventory state and the physical stock layout directly affects fulfillment reliability, slotting efficiency, and overall output.

To close this gap, we target a compact, battery–powered autonomous mobile robot that can drive through standard warehouse aisles, scan vertical rack faces up to about 2 m in height, and capture clear visual and QR–code data for every storage location. The rover is expected to work safely around people and existing equipment, and to deliver consistent, measurable performance that is suitable for real operational use.

Concretely, the system is required to:

- Navigate and align with racks with horizontal accuracy better than ± 10 cm, and position its camera vertically within ± 2 cm of the target level.
- Complete a full scan of a 1.8 m rack segment in under 3 minutes, using images with a resolution of 1920×1080.
- Reliably detect and decode 5 cm×5 cm QR codes with a success rate above 90% and maintain scanning repeatability above 95%.
- Operate within defined safety limits, including obstacle detection, collision avoidance, and an emergency stop that can bring the robot to a halt in under 500 ms.

1.2 Solution Summary

This work introduces an Autonomous Mobile Robot (AMR) specifically designed to carry out rack-face inventory verification. The rover drives itself through the aisles, aligns in front of a selected bay, and uses a vertical motion stage to position a camera at each storage level. At every stop, it reads QR codes attached to locations and links them to precise spatial coordinates within the warehouse.

The core idea is to treat the rack face as an active sensing surface. The rover combines horizontal navigation, vertical positioning, and on-board perception to maintain a structured digital representation of rack locations. Inventory verification is executed as short, repeated passes over selected rack segments rather than occasional full manual counts.

At a high level, the system is designed to achieve the following capabilities:

- **Autonomous aisle navigation:** Navigate between rack bays within a structured warehouse map without continuous human supervision.
- **Rack-level positioning:** Position the camera at variable rack heights using a controlled Z-axis mechanism that maintains alignment with the storage levels.

- **Label acquisition and decoding:** Detect and decode QR codes or location labels under typical indoor warehouse lighting and viewing distances.
- **Spatially anchored records:** Associate each decoded label with a specific physical location and time stamp so that observations can be integrated into a persistent digital rack model.

The system is engineered for deployment in real warehouse environments. The rover integrates mechanical, electrical, and software subsystems around a focused warehouse use case and is built primarily from commercially available components and open-source tools to keep the design maintainable, serviceable, and reproducible across sites.

1.3 Key Metrics and Innovation Highlights

The overall aim of the project is to deploy an autonomous system for reliable rack-level inventory verification in operational warehouse environments. The main technical objectives are:

- **Reliable navigation in aisles:** Develop a mobile base that can localise itself within a structured warehouse map and follow planned paths in narrow aisles with sufficient accuracy to align its camera with the front of the racks.
- **Controlled vertical scanning:** Design and implement a Z-axis mechanism that can move the camera smoothly from floor level up to around 1.8 m, stopping at well-defined scan heights that correspond to rack levels, while keeping vibration low enough to obtain clear images.
- **Robust data capture:** Build a perception pipeline that can reliably detect and decode QR codes at rack positions under typical indoor lighting and viewing distances, and update the inventory record with a repeatability better than 95% over repeated runs.
- **Integrated safety behaviour:** Incorporate obstacle detection and an emergency stop chain so that the rover reacts safely to people and objects in its path, and can recover to a known state after interruptions without requiring a full manual re-alignment of the mission.

These objectives are selected to align directly with practical warehouse constraints. If the robot cannot align with racks, cannot reach the required heights, or cannot be trusted to read labels accurately and behave safely, it does not address the underlying inventory problem, irrespective of the sophistication of the individual technologies involved.

Chapter 2

Problem Understanding & Motivation

Large modern warehouses often look well organised from the outside, but inside it is hard to keep the digital inventory in line with what is actually on the racks. Items are moved, partially picked, put back on the wrong shelf or mislabelled when operators are under time pressure. In most sites, people still walk the aisles with handheld scanners or clipboards and run cycle counts late at night or during low activity periods.

This manual approach does not scale well as operations grow. Its main limitations are:

- **High reliance on physical labour:** A lot of staff time goes into repetitive, low value counting work.
- **Increased risk of errors:** Tired or rushed operators are more likely to skip locations or record wrong quantities.
- **Safety and ergonomics issues:** Workers often reach into high or awkward rack positions to verify stock.
- **Out of date information:** By the time counts are completed and entered, records often lag behind the real state of the racks.

Managers then have to make decisions based on records that do not always match what is physically present. The gap between the Warehouse Management System (WMS) and the actual inventory becomes a constant source of inefficiency and slowly reduces trust in the system.

Our work is driven by this gap. We focus on one clear question that every warehouse needs to answer with confidence: **what is stored at each rack location, right now?** Instead of adding more people to do more counting, we explore whether a compact autonomous system can perform this verification task automatically, in a continuous and repeatable way.

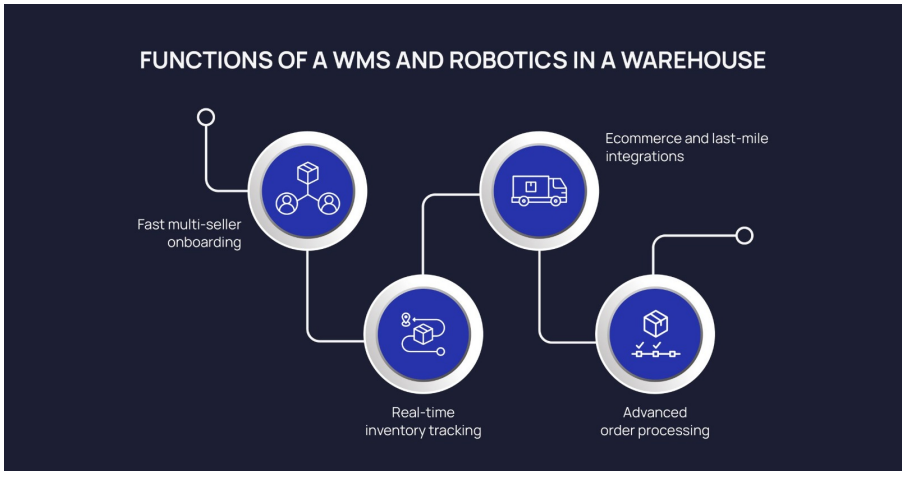


Figure 2.1: Overview of key operational stages in warehouse logistics.
Source: https://anchanto.com/wms-robotics-for-efficient-warehouse-operations/

Chapter 3

System Architecture

3.1 Block diagram (Hardware & Software)

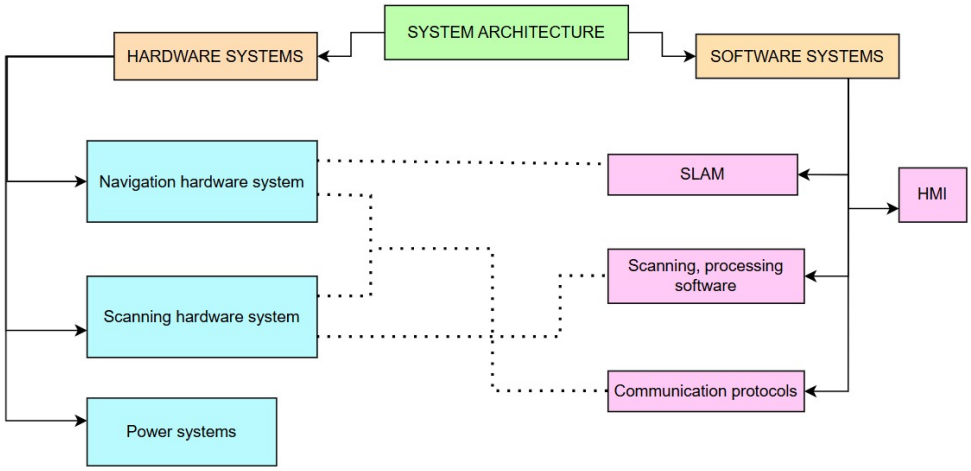


Figure 3.1: High level interaction between the rover's hardware subsystems and software stack.

3.2 Hardware subsystems

3.2.1 Navigation and Positioning

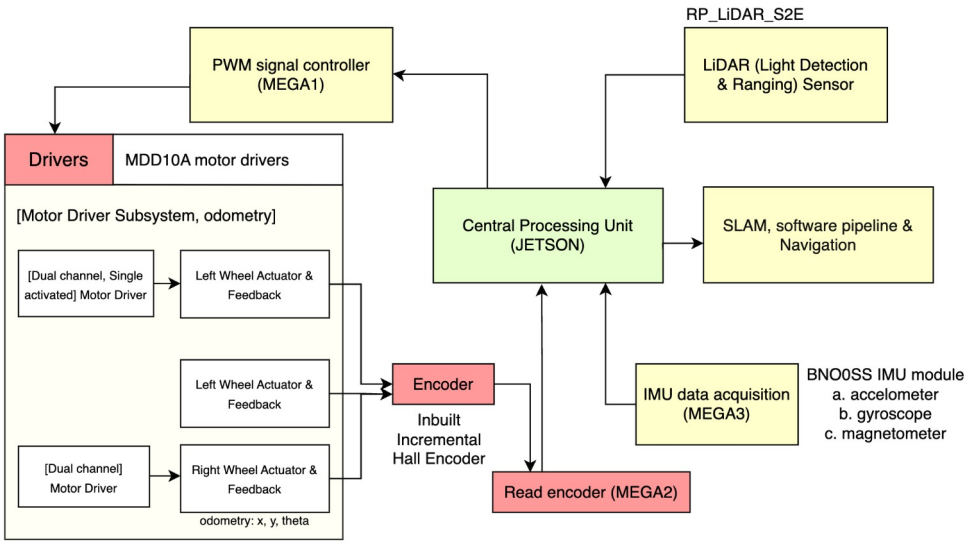


Figure 3.2: Block diagram of the robot's navigation and control hardware architecture.

X–Y Motion Subsystem

The X–Y motion subsystem is responsible for all movement in the horizontal plane. It uses a two-wheel differential drive with one caster at the front and one at the back so that the robot can move forward, backward, and rotate on the spot inside narrow aisles.

- **Actuators and drive:** Two high-torque IG42 geared DC motors drive the main wheels on either side of the chassis. The wheels are HD-type with thick rubber grip to provide sufficient friction on smooth warehouse floors and to reduce slip during starts, stops, and turns.
- **Encoders and closed-loop control:** Each IG42 motor includes an integrated rotary encoder. The Arduino Mega reads these encoder signals and uses them to control the wheel speeds in closed loop. This allows the $\approx 25\,\text{kg}$ robot to track commanded velocities reliably and to perform controlled rotations around its centre.
- **Motor drivers and power interface:** The drive motors are powered through a Cytron MDD10 dual-channel motor driver. The Arduino sends PWM and direction signals to the driver, and the driver supplies the required current to the motors. This separates power handling from control logic and keeps the motors within safe operating limits.
- **Interface to the navigation stack:** The Arduino computes odometry from the wheel encoders and sends it back to the Jetson Orin over the serial link. On the Jetson, the localisation and navigation modules fuse this odometry with data from the LiDAR, and IMU measurements that include linear acceleration, angular velocity, and yaw rate. This multi-sensor fusion forms a robust perception pipeline:
  Encoder Odometry → IMU Motion Refinement → LiDAR Based Scan Matching → Final Position Adjustment + Estimation.
  The resulting estimate is then used by the navigation stack to plan paths that respect the robot's footprint, aisle geometry, and collision constraints.

Z Motion Subsystem

The Z motion subsystem is responsible for moving the camera vertically up and down, and scanning the QR codes in real time so that different rack levels can be scanned reliably.

- **Actuation and transmission:** Vertical motion is provided by a high-torque IG45 geared DC motor that drives a continuous timing belt. The belt runs over a set of pulleys and is fixed to the camera carriage. The pulley layout keeps the belt in a single plane, which reduces side loads on the guides and lowers the torque required from the motor.
- **Direction of motion:** Clockwise rotation of the IG45 motor drives the camera carriage downward, while counter-clockwise rotation moves it upward. The mapping between motor rotation and vertical travel is governed by the pulley diameter and belt pitch, and this relationship is used when computing precise target heights for each scan position. Additionally, the X-channel structural base provides lateral rigidity to the entire assembly, significantly reducing wobble during vertical motion and ensuring stable positioning at all rack levels.
- **Motor driver and control interface:** The IG45 motor is powered by a Cytron MDD10A motor driver. The Arduino Mega, same as the one used in drive, sends PWM and direction signals to this driver, following the same pattern as the X–Y drive. This keeps the control interface consistent across the motion subsystems.

- **Z Position feedback:** The IG45 motor includes an integrated encoder that provides position feedback. The encoder ticks are linearly mapped to the camera height. Based on experimental results, 163 encoder ticks correspond to 1 cm Z motion. This information is used by the scanning algorithm to obtain the position of the QR scanned.

3.2.2 Scanning

Camera unit

An IMX258 13MP OIS USB Camera module has been mounted on a carriage on the vertical mechanism, driven by the timing belt. The camera subsystem can further be divided into two parts – actuation and capture.

- Actuation – A couple of SC15 servo motors have been mounted on the carriage, providing two extra degrees of freedom to the camera module, allowing clear scans in case the camera is not able to reliably detect or interpret a QR code at the extreme ends of its FOV. These motors are controlled using Waveshare ESP32 servo expansion board which is also mounted on the carriage. This driver precisely controls and returns the position feedback of the two servos, effectively giving the exact position and orientation of the camera module.
- Capture – Once the servo motors and the vertical mechanism have stably reached the final position and orientation, the vision pipeline first detects and then scans the QR codes within its current FOV, giving the relevant output on the HMI.

3.2.3 Power

The following flowchart illustrates the power subsystem. It clearly indicates how the power is distributed from a centralised battery pack to every subsystem. The battery supplies power parallely to four power rails– A 7V power rail powered through a 12-7 step down buck converter to power the arduino megas and the servo driver, a stable 12V rail powered by another 12-12 buck converter to power jetson orin and LiDAR, and two 12V rails directly from the battery, secured by a 10A fuse to power the motor drivers.

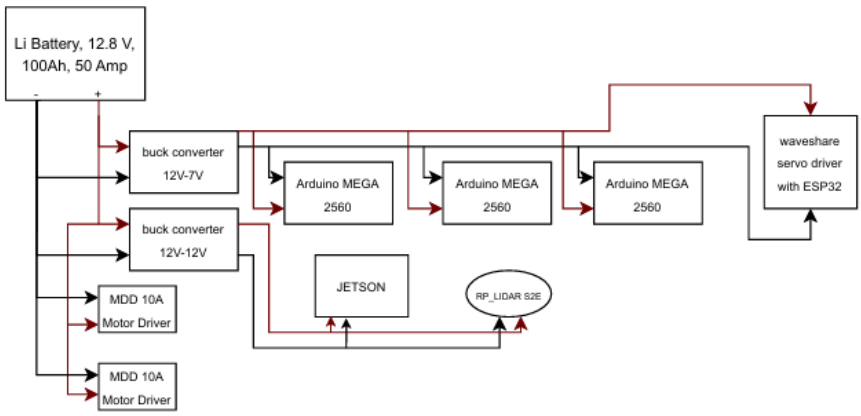


Figure 3.3: Power distribution diagram for battery, buck converters, and controllers.

3.3 Software Stack

3.3.1 SLAM

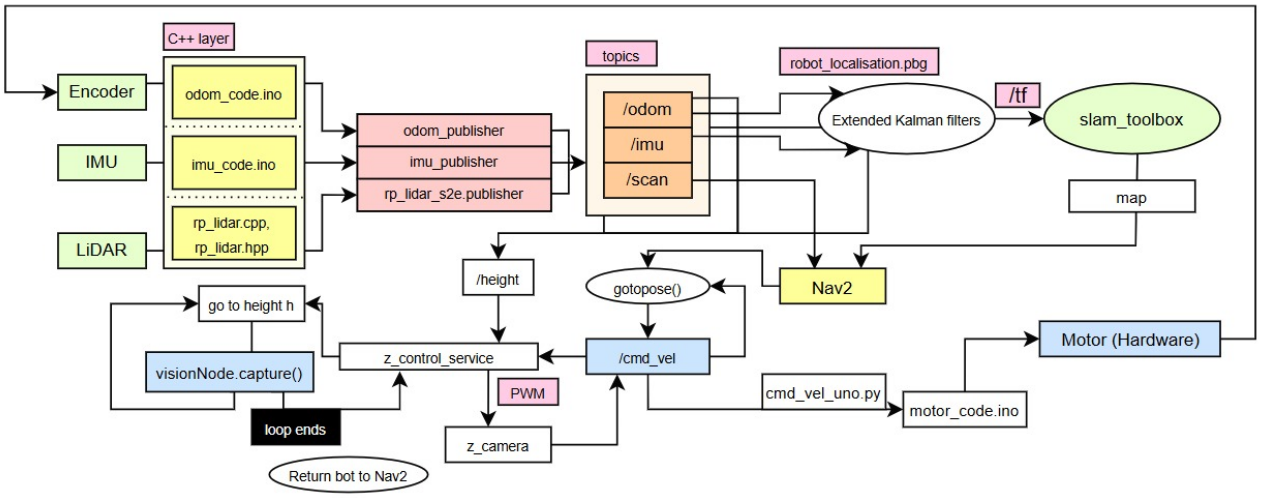


Figure 3.4: Software pipeline from sensors and vision node to SLAM, Nav2, and motor control.

Simultaneous Localisation and Mapping (SLAM) allows the rover to build a map of the warehouse while at the same time and position estimating its own pose inside that map. Using only onboard sensors, the system observes surrounding walls and racks, tracks how the base moves, and produces a 2D occupancy grid that represents free, unknown and occupied space.

On this robot, SLAM Toolbox performs this task by consuming LiDAR scans together with IMU and wheel odometry information. As the rover moves, the node incrementally updates the map and corrects drift by aligning new scans with previous ones using *robot_localisation* which fuses data of the two sensors that is odometry and IMU. The resulting map and pose estimate are then used by the Nav2 stack as the global reference for path planning, obstacle checking and repeated autonomous missions.

3.3.2 QR

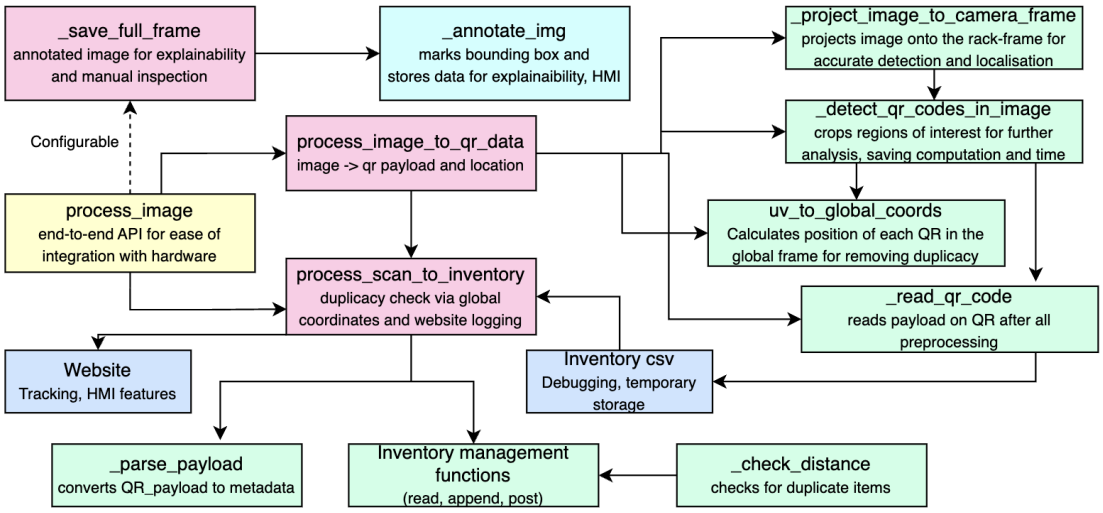


Figure 3.5: Complete Vision Pipeline from raw image to data logging

The complete QR processing and data logging pipeline is present in Processor.py and outlined above with function names. We project images onto the rack frame -> Extract snapshots of QRs from it -> Calculate location and payload of each -> log all detections to Inventory and eliminate duplicates -> Send the data to website. Human Interaction with the machine occurs through the website, and command line entry points in the code.

3.3.3 HMI

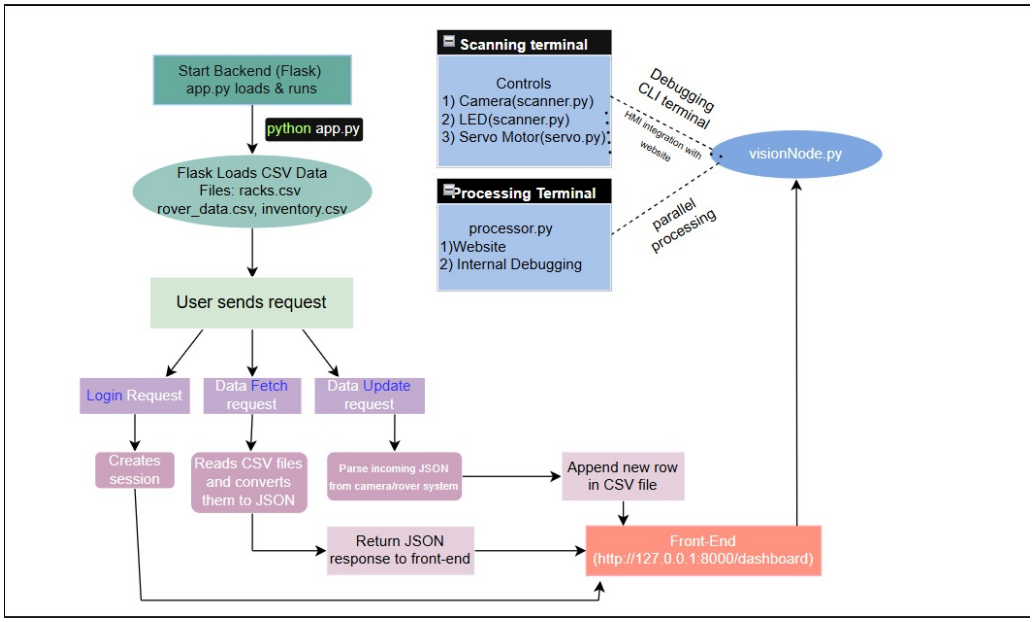


Figure 3.6: Block diagram of the Human Machine Interface

Website Interface

The human machine interface (HMI) for the rover is implemented as a small web application that runs on top of the core ROS 2 navigation and perception stack. The HMI has two main roles:

- provide operators with a live view of rack states, robot status, and recent scans, and
- expose a simple control surface for starting, stopping and monitoring runs without logging into the robot over SSH.
- manually control the movements of the camera precisely to enhance field of view of the scanning process

The web application is built from two HTML templates, *login.html* and *dashboard.html*. The login page restricts access to authorised users. After successful authentication it redirects to a dashboard that shows a list of rack cards, each card displaying the shelf identifier along with the code of the item present in the shelf. The event logs in below the live rack keeps track of all the scanning activities and motion of the bot along with the corresponding timestamp.[1]

The back end is a Python Flask service that runs in a separate *backend* folder. A single file *app.py* holds the Flask application, session based authentication logic, CSV parsing functions and HTTP endpoints for the dashboard. The service reads and updates three CSV files: *rover_data.csv* for telemetry, *racks.csv* for rack metadata and *inventory.csv* for inventory records. These CSV files act as a light data store for the HMI and keep the web layer independent of the ROS 2 log formats.[2]

At run time the perception nodes write scan results and inventory events into the CSV files. The Flask back end normalises these entries and serves them as JSON to the dashboard, which periodically requests updates. In this way the HMI stays in sync with the robot without affecting the timing of the core navigation and scanning pipelines.

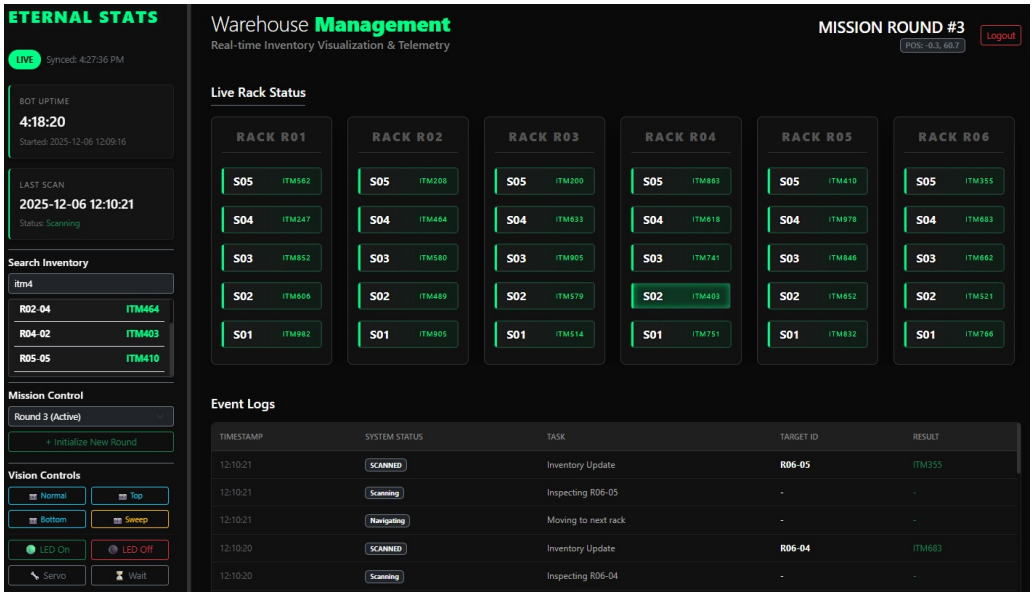


Figure 3.7: Interface of website

[1] Front end and back end structure follow the separate website documentation supplied with the project.
[2] Backend organisation and file layout are taken from the website documentation.

Command Line Interface

Along with the web dashboard, a light command line interface is provided for developers and maintenance staff. It exposes a few focused tools that help in debugging, parameter tuning and checking the health of the vision hardware.

- ***save_annotated_image* flag:** A flag in *compile.yaml* lets the user enable saving of annotated images. When it is set, the pipeline stores projected frames with QR boxes, confidence scores and basic metadata at a user specified path. These images are used to see what the detector is actually picking up and to tune model thresholds for a given warehouse.
- **Servo angle display and *return_to_home*:** A small utility prints the current angles of the pan and tilt servos that control the camera. The code also exposes a *return_to_home* helper where an operator can record the neutral values of *Servo[0]* and *Servo[1]*. At the end of a session the camera is sent back to this neutral pose, which compensates for mechanical drift over time and keeps the motion range within safe limits.

3.4 Data flow diagram

The data path from sensors to the HMI proceeds in two main streams. Encoder, IMU and LiDAR signals are first read by the motor and sensor controllers, which send compact messages to the Jetson. On the Jetson, ROS 2 nodes convert these messages into the */odom*, */imu* and */scan* topics. These topics are fused by the state estimation node and passed to *slam_toolbox*, which maintains the 2D map and provides the robot pose that is used by the navigation stack. The same pose and velocity commands are forwarded to the Arduino MEGAs that drive the base and lift motors.

In parallel, the camera stream is processed by the vision node. For each scan request it captures an image, runs the QR detection and decoding pipeline, and writes the decoded item ID together with rack, shelf and global coordinates into an inventory table. A lightweight Python back end reads this table, exports it as JSON, and serves it to the web based HMI. The HMI dashboard then presents live rack status and a log of scan events to the operator.

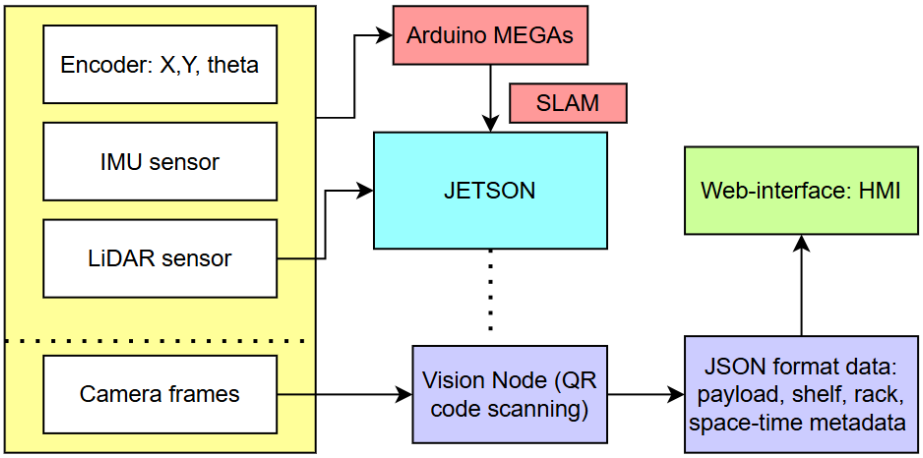


Figure 3.8: Block diagram of the Data flow from the sensors to the HMI

Chapter 4

Hardware Design

4.1 Chassis Specifications

The mechanical layout of the base is designed around a compact footprint and stable support for the X–Y and Z motion subsystems.

- **Footprint and support:** The chassis is dimensioned to stay within the $600\,\text{mm} \times 450\,\text{mm}$ constraint while carrying the full system mass of approximately $25\,\text{kg}$. Two drive wheels are mounted on the sides and two casters are placed at the front and rear, giving a four-point support that spreads the load and maintains stability.
- **Commodities and raw material selection:** The base plate has been fabricated using a 3 mm aluminium plate to provide the required strength to bear the load of the 25 Kg bot. All other plates have been fabricated using 3 mm acrylic to reduce the total weight of the bot. The plates are supported using 2020 aluminium extrusions, which have low density, high tensile and compressive strength, and high durability. M5 fasteners have primarily been used in most of the joints. M4 and M3 fasteners have been used wherever necessary in case of mounts and hubs. Mild steel shafts have been used for rotatory components to ensure stability and sturdiness to the bot as many of these are meant to bear very high load.
- **Safety oriented chassis geometry:** The chassis has been designed keeping in mind ISO and RIA standards. The base plate has been thoroughly chamfered to avoid any tips or pointed edges. All the edges have been covered by aluminium extrusions having smooth edges to even further enhance the safety of the people around the bot.

4.2 Drive System

- **Differential drive geometry:** The drive wheels and omni casters are positioned so that the centre of rotation lies close to the geometric centre of the robot. This allows the base to rotate almost in place inside an aisle without the corners moving outside the permitted clearance near the racks.
- **Drive wheel selection and floor interaction:** The powered wheels are HD-type wheels with rubber grips. The combination of wheel diameter, tread material and motor torque is chosen to provide reliable traction on smooth warehouse floors while keeping vibration and slip low during starts, stops and turns.
- **Omni wheel selection:** Two omni wheels, one at the front and one at the back, act as stabilizers. They are mounted on 8 mm shafts with bearings and maintain consistent contact with floor, keeping the robot balanced in all directions. Their diameter matches that of the drive wheels, ensuring all four wheels share the load and remain in contact with ground.
- **Motor control unit:** The motor control unit connects the Arduino Mega PWM and direction pins to two Cytron MDD10A motor drivers. The Jetson sends velocity and Z-axis position commands over serial, and the Mega converts these into PWM signals that drive the IG42 differential base motors and the IG45 Z-axis actuator.

4.3 Vertical Lift Mechanism

The Z-axis mechanism provides the structural and actuation system for the vertical motion of the camera and scanning assembly.

- **Belt–pulley layout:** The camera carriage runs on a rigid vertical frame with guided rollers for stable, repeatable motion. A continuous timing belt operates in a single plane over three pulleys (motor, top idler, in-line idler), minimising twist, side loads, and uneven tension.
- **Load distribution and torque reduction:** The single-plane belt routing limits off-axis forces and improves mechanical efficiency, reducing motor torque and power consumption. A long bolt functions as a passive idler to maintain uniform belt tension across the entire stroke.
- **Constant pull mechanism:** The belt arrangement ensures the carriage always experiences a controlled driving force in both upward and downward directions, improving reliability compared to friction-dependent belt systems.
- **Motion characteristics:** Pulley diameters and belt pitch define the conversion between motor rotation and vertical travel. These parameters establish discrete scan heights in software and guarantee full coverage from floor level to the highest rack position.
- **Structural stiffness:** The belt, pulleys, and carriage are mounted on aluminium extrusion profiles fastened at multiple points on the base, providing the rigidity required to minimise deflection, maintain rack alignment, and ensure consistent imaging.
- **Constant stable actuation:** Due to the rigidity of the Z-axis assembly, closed-loop control is unnecessary. A constant PWM drives the motor until the target height is reached. Predefined positions based on camera FOV and rack geometry enable fast, stable vertical positioning with minimal computational load.

4.4 Sensor Justification

The main control unit is an NVIDIA Jetson Orin Nano running the ROS 2 stack for localisation, mapping, planning and scanning. Arduino Mega 2560 boards and an ESP-based servo controller handle direct hardware control, with commands and feedback exchanged over UART.

Feedback Units

The feedback electronics are organised into three functional units:

- **IMU unit:** Interfaces a BNO055 absolute orientation sensor to an Arduino Mega over I^2C. This 9-DoF sensor provides highly accurate orientation and acceleration data. Unlike other IMU modules, BNO055 consists of onboard sensor fusion, directly generating quaternions, eliminating the need for an external filter. The Mega reads the IMU metadata and forwards quaternion, accelerometer and gyroscope data to ROS 2 nodes on the Jetson via serial communication.
- **Odometry unit:** Connects the incremental encoders of the X, Y and Z drive motors to digital inputs on an Arduino Mega through interface pins. Motors with inbuilt encoders have specifically been used, primarily to save space and reduce the need for external encoders. Moreover, they require less power and since they need not be externally mounted or coupled to the motor shaft, they provide accurate motor shaft rotation and odometry data. The

encoder counts are converted into drive and lift odometry and sent to the navigation stack as position and orientation estimates.

- **LiDAR unit:** Scans the surroundings using a 360-degree laser sweep in the scanning plane. The RPLiDAR S2E provides a range of upto 30 m radially with a sampling frequency of over 10 KHz, ideal for a warehouse or dark store. It has also got an eye-safe laser, providing a safe environment for workers working in proximity of the bot. Most importantly, it is connected to Jetson using Ethernet instead of conventional USB communication, providing high bandwidth, less noise, easy integration with ROS and ROS2, and significantly enhancing data transfer speed. The pointcloud generated is integrated with the odometry and imu readings using the SLAM toolbox to create a well-defined map of the surroundings.

The ESP based servo controller drives the pan–tilt camera mount. The Jetson Orin sends orientation targets, and the ESP converts these targets into low level servo commands that move the camera to the desired pose.

This architecture keeps high level decisions on the Jetson Orin Nano and delegates timing sensitive drive, encoder and sensor handling to microcontroller based units.

Vision and Sensing Units

The vision and sensing layer combines the SC15 servo bus and the main camera module to give stable pointing and reliable image capture for the QR pipeline.

- **SC15 position feedback:** The SC15 bus servo provides angle and load feedback through its own encoders. It holds position with minimal backlash and good repeatability, which keeps the camera pose stable while frames are captured.
- **Camera module:** The camera has optical image stabilisation, a wide horizontal and vertical field of view and hardware autofocus (not used in our current setup). It connects over USB, which works well for longer cable runs inside the robot. The module is compact and light, and integrates cleanly with the Jetson Orin Nano for real time processing.

4.5 Power Systems

The PDB routes power from the main 12.8 V battery to the subsystems and generates separate regulated rails for compute, control, sensing and feedback devices.

- **Battery input:** A 12.8 V nominal battery feeds the PDB. The wiring and connectors on this input are rated for up to 50 A continuous current, covering expected peaks during base acceleration and Z-axis motion.
- **High-current motor branch:** The two Cytron MDD10A motor drivers (for the IG42 base motors and the IG45 Z-axis motor) are supplied directly from the 12.8 V battery bus through a series fuse.
- **Logic and servo rail (7 V buck):** A 12 A DC–DC buck converter generates a regulated 7 V rail for low-voltage control electronics:
  - three Arduino Mega boards for motion control and I/O,
  - one ESP-based servo controller for the pan–tilt unit, and
  - auxiliary loads such as the IMU and status indicators via 5 V / 3.3 V regulators.
- **Compute and sensing rail (12 V buck):** A second 12 A buck converter provides a reg-

ulated 12 V rail for the Jetson Orin Nano and the 2D LiDAR. This branch is protected by a series fuse sized for an expected combined current of up to 2 A.

- **Load summary by rail:** Table 4.1 summarises the main power rails and their associated loads.

Table 4.1: Power rail summary on the PDB

Rail	Nominal rating	Primary loads
Battery bus	12.8 V, up to 50 A	2 × Cytron MDD10A motor drivers (IG42 base motors, IG45 Z-axis motor)
Buck 1	7 V, 12 A	3 × Arduino Mega, 1 × ESP servo controller, IMU, status LEDs/buzzer
Buck 2	12 V, 12 A	Jetson Orin Nano, LiDAR (via fused branch, $\leq$2 A typical)

All high-current paths (battery to PDB, PDB to motor drivers and PDB to buck converters) are fused and use appropriately sized conductors and crimped terminals. Logic, encoder and compute grounds are tied to the main power ground at a single reference point on the PDB to limit ground loops and reduce noise on sensor and communication lines.

4.6 Safety Features

Safety features are built into the motion architecture at both hardware and software levels.

- **Controlled speeds and torques:** Wheel and lift commands are limited in software to stay within safe velocity and acceleration bounds, reducing the risk of instability when the robot rotates in place or when the Z-axis is near its upper positions.
- **Closed-loop monitoring:** Encoders on the X–Y motors allow the controller to detect deviations from expected motion, such as stalls or unexpected resistance.
- **Z motion limits:** Physical limit switches have been attached at the top and bottom of the vertical rail which prevent overextension of the camera carriage beyond desired limits. Set in the NC configuration, these switches break the motor circuit when pressed.
- **Integration with higher-level safety logic:** The Jetson Orin applies motion constraints based on obstacle detection, mission state, and safety inputs. The robot includes emergency-stop buttons on the both sides, completely complying with the ISO standards. It has also got status LEDs and an active buzzer for bot status and warnings.
- **Motion enforcement:** The low-level controller monitors safety peripherals and enforces ISO-aligned operational rules, including speed limits in shared zones, controlled deceleration near humans, and mandatory halts on critical faults. When the perception layer detects a safety event, the Jetson Orin issues an immediate stop or restriction command, which the Arduino applies deterministically through the motor drivers.

Chapter 5

Software Architecture

5.1 Navigation Pipeline

The navigation pipeline is a ROS 2 based stack that converts raw sensor data into safe velocity commands for the base. It is organised into four stages:

- perception and state estimation,
- SLAM and global map building,
- path planning and local control,
- obstacle avoidance and recovery behaviours.

Perception combines 2D LiDAR, IMU and wheel odometry. An Extended Kalman Filter (EKF) fuses IMU and encoder data to provide a smooth estimate of pose and velocity in the *odom* frame. SLAM uses this estimate and the LiDAR scans to build a 2D occupancy map in the *map* frame. Nav2 then plans and executes motion using the map, the current pose and live costmaps. All components communicate through ROS 2 topics, services and TF transforms such as *map*, *odom*, *base_link* and *laser*.

5.1.1 SLAM

The SLAM module maintains a consistent 2D map of the environment while estimating the robot pose in that map.

Sensors and inputs

- **LiDAR:** publishes 2D laser scans on */scan*.
- **IMU:** publishes orientation and angular velocity on */imu*.
- **Wheel odometry:** publishes integrated pose from encoders (for example on */odom/wheel*).
- **EKF node:** fuses IMU and wheel odometry into a filtered odometry estimate on */odom* and maintains the *odom* to *base_link* transform.

SLAM Toolbox node

The *async_slam_toolbox_node* subscribes to the LiDAR scans on */scan* and to the robot pose in the *odom* frame via TF. It incrementally builds a 2D occupancy grid map on the */map* topic and can publish incremental updates on */map_updates*. It also publishes the *map* to *odom* transform, correcting long term odometry drift through scan matching and loop closure.

TF frame chain.

The resulting TF tree has the form *map* → *odom* → *base_link* → *laser*, where:

- *map* → *odom* is provided by SLAM Toolbox,

- *odom* → *base_link* is provided by the EKF,
- *base_link* → *laser* is a static transform that encodes the LiDAR mounting pose.

At any time the global pose of the robot is obtained from the compound transform *map* → *base_link*, which is what Nav2 uses for planning. The map is built online as the robot moves. New areas are added and previous areas are refined when the robot revisits them.

5.1.2 Planning

Planning is handled by the ROS 2 Navigation2 (Nav2) stack. It converts high level goal poses into velocity commands that respect the global map and local obstacles.

Inputs to planning

- **Global map:** occupancy grid from SLAM Toolbox on */map*.
- **Robot pose:** obtained from the TF chain *map* to *base_link*.
- **Sensor data:** live LiDAR scans used to update global and local costmaps.
- **Goal pose:** a 2D pose in the *map* frame, set in RViz or by an external node.

Global planning

The global planner computes a collision free path from the current pose to the goal using the global costmap. The path is represented as a sequence of waypoints in the *map* frame.

Local planning

Nav2 incorporates its own internal PID style control inside the local planner. The local planner runs at a higher rate using a local costmap centred on the robot and:

- continuously computes heading and position error relative to the planned path and adjusts linear and angular velocities to correct these errors,
- scales the commanded speed based on path curvature, proximity to obstacles and dynamic limits defined in *nav2_params.yaml*,
- produces a stabilised velocity command (*geometry_msgs/Twist* on */cmd_vel*) without requiring any external PID node.

This internal control loop performs the necessary regulation so that the robot tracks the planned trajectory smoothly while respecting safety and kinematic limits.

Controller

Within Nav2 the closed loop control is handled by the local controller rather than by a separate PID node. The global NavFn planner first computes a path in the map frame using the A* algorithm instead of plain Dijkstra, which keeps the path length short while respecting the global costmap.

The local DWB controller then tries to follow this path by sampling velocity commands around the nominal *cmd_vel* and scoring them against the costmaps and kinematic limits. Because the real

robot is not perfectly symmetric and the two drive motors do not reach exactly the same maximum speed, the base tends to deviate from the ideal trajectory. The DWB controller continuously evaluates the tracking error from odometry and selects the command that best reduces this deviation while remaining collision free. In practice this provides the role of a PID style stabilising loop, but fully integrated inside the Nav2 control layer.

Behaviour tree coordination

Nav2 uses a behaviour tree to coordinate tasks such as computing the path to a pose, following the path and running recovery behaviours like clearing costmaps, rotating in place or replanning when the path is blocked.

5.1.3 Avoidance

Obstacle avoidance is configured through the *nav2_params.yaml* file, which sets how costmaps are built, how obstacles are inflated and how often the environment is refreshed. The behaviour is implemented through costmap layers, continuous sensor updates and the local planner.

Costmap layers.

Nav2 maintains two main costmaps:

- **Global costmap:** built from the SLAM map in the *map* frame.
- **Local costmap:** a robot centred costmap that is updated at a higher rate from live LiDAR data.

Both costmaps use layered plugins:

- **Static layer:** loads the occupancy grid produced by SLAM Toolbox.
- **Obstacle layer:** inserts and clears obstacles based on incoming LiDAR scans.
- **Inflation layer:** expands obstacle cells to create a safety margin around the robot footprint.

Online obstacle detection.

The LiDAR sensor continuously publishes laser scans on */scan*. These scans update the obstacle layer in the local and global costmaps. Free space is marked when beams reach the background, and occupied cells are marked when returns are received from objects. As a result the costmaps reflect the current layout of racks, walls and temporary obstacles.

Reactive avoidance.

The local planner reads the live local costmap and generates */cmd_vel* commands that:

- steer the robot away from cells with high cost that correspond to obstacles,
- adjust linear and angular velocity as the robot approaches obstacles,
- follow the global path while maintaining a safe clearance.

If no feasible command can be found, the behaviour tree triggers recovery actions such as rotating

to rescan the area, clearing costmaps and requesting a new global path from the current pose.

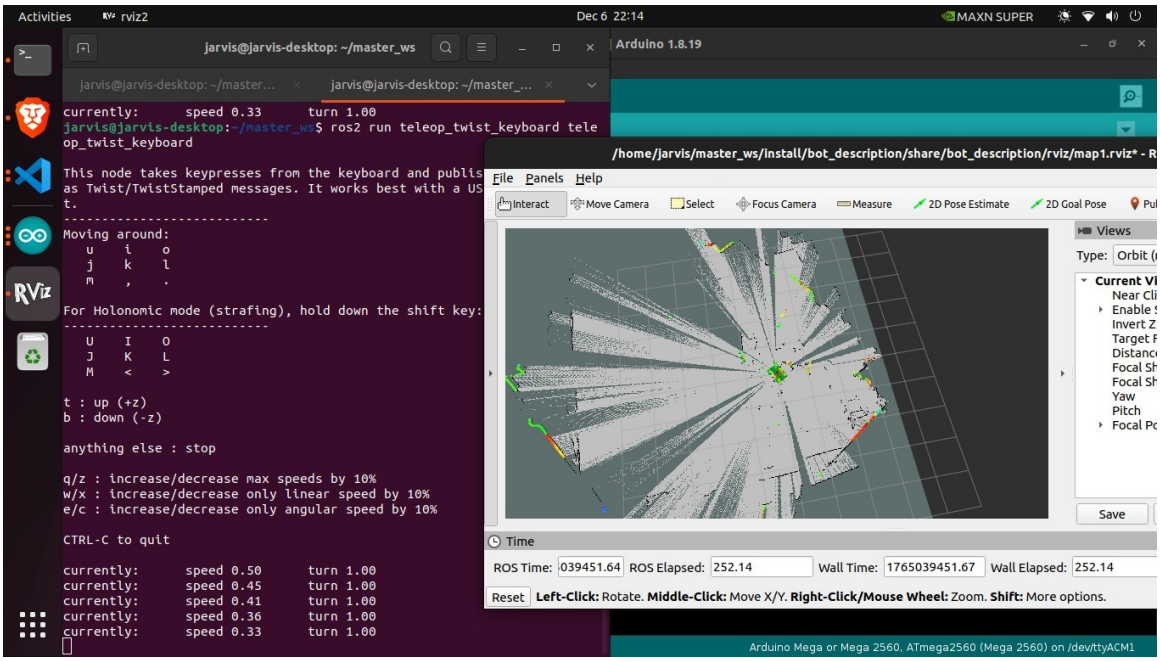


Figure 5.1: Mapping Using SLAM

5.2 Vision Pipeline

The vision subsystem forms the core of our warehouse inventory workflow. It performs real-time detection, decoding, and localization of QR codes(and hence the Items) placed on shelves, enabling the robot to map inventory items into a global coordinate frame integrated with the SLAM module. The complete scanning pipeline spans hardware, software orchestration, multiprocessing data flow, and a structured output layer. A high-level breakdown is presented below.

5.2.1 High-Level System Architecture

The vision stack follows a modular design with three layers:

- **Hardware Layer** Physical sensing and actuation responsible for capturing multi-angle images and indicating system state.
- **Software Layer (Python)** Execution engine that performs scanning, servo control, image processing, and QR decoding.
- **Data Layer** Structured storage for debugging outputs, detections, and synchronization.

Hardware Layer

- **USB Camera (IMX258, Waveshare Module)** High-resolution sensor used for all image captures. Provides stable frame quality through reduced motion blur during multi-angle scans.
- **Two SC15 Serial Servo Motors (Pan–Tilt Unit)** Used for orienting the camera across multiple vertical (Bottom–Normal–Top) and horizontal (Left-Normal-Right) viewpoints. Enables angle-sweep scanning for improved QR visibility.
- **ESP32-Servo driver expansion board** Consists of all the bridges and internal connections

required to drive the serial servo motors efficiently. Enables accurate position control of the servo and transmits precise angular feedback back through serial communication.

- **On-board RGB LED Controller** Two RGB LEDs integrated on the servo driver board act as:
  - Status indicators (system health, capture confirmation)
  - Motion indicators (servo movement / in-progress scanning)
  - Active led safety indicator on movable Z-axis assembly as per standards

Software Layer

The vision software is divided into an orchestrator and two multiprocessing subsystems:

- **visionNode (Orchestrator)** Implements a singleton control layer responsible for:
  - Coordinating multi-angle scan logic
  - Controlling LED signaling
  - Invoking servo movements through Scanner
  - Queue-based communication with the background Processor
- **Multiprocessing Architecture** The system follows a producer–consumer model:
  - **Scanner Module (Producer)** Captures burst images, selects the sharpest frame, logs metadata, and pushes scan jobs into a multiprocessing queue.
  - **Processor Worker (Consumer)** Continuously consumes queued scan jobs:
    * Preprocesses and rectifies images
    * Detects and decodes QR codes
    * Projects local detections to global coordinates
    * Updates inventory database
    * Sends results to remote web API server

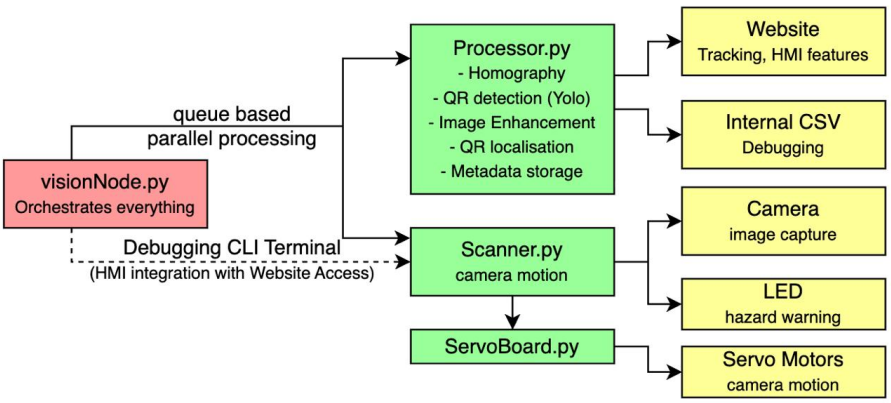


Figure 5.2: High level architecture of Vision Pipeline

Data Layer

The system outputs and stores structured data for debugging, analytics, and remote access:

- **Raw Image Archive (PNG)** Unprocessed captures saved alongside timestamped filenames.
- **Metadata CSV Logs** Contains pose angle, sharpness score, capture time, and file paths.

- **Annotated Debug Images** Visual overlays for QR bounding boxes, projections, and decode results.
- **Inventory Database (CSV)** Maintains the latest decoded QR entries mapped to global coordinates.
- **Remote API Server (HTTP POST)** Locally hosted web service where the robot pushes processed scan results for user-facing dashboards or warehouse management software.

5.2.2 Data Flow Overview

The vision data pipeline comprises three compact layers: input acquisition, multi-stage processing, and structured output generation. This section outlines how raw sensory inputs propagate through the orchestrator, scanner subsystem, background processor, and ultimately into the inventory database and remote server.

Input Layer

- The system receives three sources of data: (a) global pose offsets (x, y) from the SLAM system; (b) scan commands (*isTop*, *isBottom*) from the navigation layer; (c) raw frames from the USB camera (IMX258) accessed through the V4L2 backend. Together, these determine the orientation sequence and spatial tagging for each capture.

Processing Layer

- **The *visionNode*** acts as the central coordinator: selects orientations, commands the servos, performs LED signalling, triggers frame capture, and enqueues jobs for asynchronous processing.
- **Scanner subsystem** performs burst acquisition with sharpness-based selection. Each trigger flushes the driver buffer, captures multiple frames via fast *grab()* and *retrieve()*, computes Laplacian variance, and stores the sharpest frame with timestamped metadata.
- **Processor worker** runs in a separate process, loads the saved image, and performs the full pipeline: geometric rectification to camera-frame coordinates; QR localisation using a YOLO-based detector; QR enhancement (CLAHE, unsharp masking, thresholding) and decoding via the WeChat engine; centroid-to-global conversion using intrinsics and SLAM offsets; duplicate filtering; inventory file updates; optional annotated-image export; and HTTP transmission of structured scan results to the remote API.

Output Layer

- The pipeline outputs: a consolidated *inventory.csv* containing session ID, payload, rack/shelf identifiers, and global coordinates; optional annotated frames for debugging; raw captures (subject to deletion policy); and HTTP POST updates consumed by the warehouse management web application. These constitute the interface between onboard perception and higher-level autonomous systems.

5.2.3 Design choices and their justifications

- **Burst capture + Laplacian selection** real robots move. A single-frame capture is brittle; grabbing a short burst and selecting the sharpest image yields substantially higher decode rates for small codes at distance. That's cheap (a few ms per frame) and reliable.
- **Asynchronous worker (multiprocessing)** decoding, rectification and networking are CPU/IO heavy. Putting this work into a separate process keeps capture and servo control responsive and avoids blocking the control loop. Using file paths (instead of raw images) avoids heavy interprocess memory transfer.
- **Geometric projection before detection** QR detectors are typically trained on near-fronto-parallel images. Our projection uses physical intrinsics and the servo orientation to synthesize a front normalized view. That improves detector recall and decoder success without collecting extra training data. The camera calibration numbers confirm metric reliability (RMS ≈ 0.45, mean reprojection ≈ 0.06 px), so centroids map into global coordinates with sub-pixel accuracy.
- **Explicit intrinsics & distortion handling** distortions and principal point offsets bias centroid-to-world conversion. We correct for them up-front so the mapping from pixel to meters is physically meaningful and reproducible across deployments.
- **Minimal servo motion planning** `visionNode` tracks the current `curr_h`/`curr_v` and sequences moves to avoid unnecessary sweeps, saving time and mechanical wear. This is a simple stateful optimization with big practical upside.
- **Simple CSV inventory + Remote POST** CSV gives an auditable local store for debugging and offline inspection while the HTTP POST integrates with remote dashboards. This dual output is pragmatic: local traceability plus real-time sync.

Geometric projection and distortion handling are described in detail in the Innovation and Novel Approach section, where the elaborate justification and mathematical basis for these choices are presented.

5.2.4 Control Flow: System Initialization Sequence

The scanning pipeline is executed by the *visionNode* orchestrator, which coordinates servo positioning, image acquisition, background processing, QR decoding, and inventory updates. The complete execution loop is given below.

1. **Load Configuration (YAML)** All camera, servo, processor, and session parameters are loaded from *config.yaml*. This includes:
   - Camera parameters (resolution, FPS, codec)
   - Servo configuration (port, baud rate, angular mapping)
   - Processor thresholds (QR confidence, NMS, padding)
   - Session information (round ID, server URL)
2. **Initialize *visionNode* Singleton** Creates log directories, initializes the scanner, sets home servo position, and spawns the background processor.
3. **Scanner Subsystem Initialization**
   - Connects to camera using V4L2 (video four linux v2) backend

- Configures image properties (width/height/FPS) and FourCC codec
- Performs warm-up reads
- Initializes servo hardware (*servoBoard*)

4. **Launch Processor Worker (On multiprocessing queue)** The worker process continuously waits for incoming tasks on a shared queue.
   - Queue capacity = 50 tasks
   - Each task contains: image path, SLAM offsets, camera orientation
5. **System Ready** Scanner is active, servos are at neutral orientation, and the processor is ready to accept images.

5.2.5 Scanner Subsystem: Optimized Frame Acquisition

The *Scanner* module is responsible for all low-level image acquisition tasks. Its design ensures that each scan request produces the sharpest possible frame, even under vibration, robot motion, or varying lighting conditions. Instead of relying on a single capture, the scanner performs a structured burst-capture and post-selection workflow. This significantly improves the quality of downstream QR detection and geometric rectification.

Burst Capture and Sharpness-Based Selection

The full internal workflow is as follows:

1. **Burst Capture (10 frames)**
   - Flush camera buffer by discarding the first 3 frames (eliminates stale frames from the driver queue)
   - Use *grab()* for fast frame acquisition (no immediate decoding, hence minimal latency)
   - Use *retrieve()* to decode only the frames that were grabbed
2. **Sharpness Analysis** For each retrieved frame:
   - Convert the frame to grayscale
   - Apply the Laplacian operator to estimate edge strength
   - Compute the variance of the Laplacian response (a higher value indicates a sharper image)
   - Track and select the frame with the highest sharpness score
3. **Persisting the Best Frame** Once the sharpest frame is identified:
   - The image is saved with a timestamp-based name: *capture_YYYY-MM-DD_HH-MM-SS.png*
   - Files are stored under the configured raw-capture directory (e.g. */data/raw_captures/*)
   - A metadata entry containing timestamp, filename, resolution, and sharpness score is appended to a CSV log

This subsystem ensures that every capture request produces a clean, stable, and high-quality input image. These saved images are then asynchronously consumed by the *processor_worker*, which performs geometric rectification, QR detection, decoding, and global coordinate mapping.

5.2.6 QR Detection Pipeline

QR detection in the system follows a clear multi stage pipeline built for warehouse conditions such as changing light, narrow aisles and a moving platform. Each step cleans the image and refines the candidates so that the final detections stay reliable.
The entire pipeline has two major sections

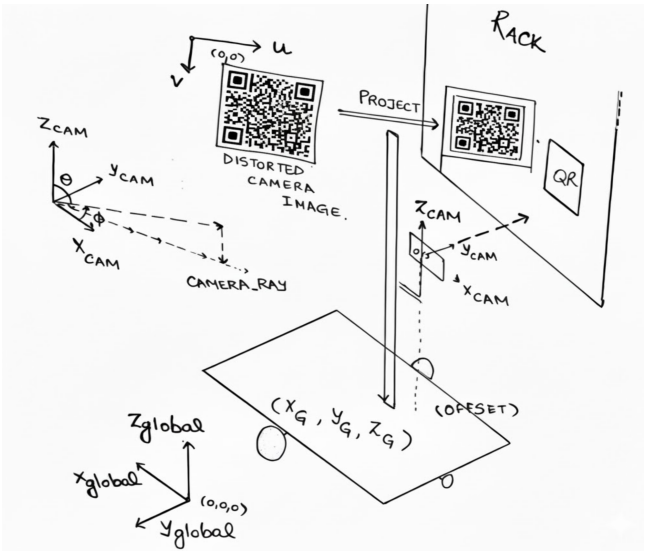


Figure 5.3: We use the above three coordinate frames to establish a 3D projection of image to a plane parallel to the rack (Geometric Rectification)

A) Geometric Rectification

1. Convert orientation (0,1,2) into camera pitch/yaw angles using the orientation map defined in the processor module

$$\text{Down} \rightarrow (180^\circ, 90^\circ), \quad \text{Front} \rightarrow (90^\circ, 90^\circ), \quad \text{Up} \rightarrow (0^\circ, 90^\circ)$$

2. Compute rotation matrix R using camera model:

$$R = [x_{\text{cam}}, y_{\text{cam}}, z_{\text{cam}}]$$

3. For each pixel (u, v), obtain the undistorted ray direction:

$$\vec{r}_{world} = R \cdot K^{-1}(u, v)$$

4. Intersect this ray with the warehouse wall plane at distance d to obtain real-world surface coordinates:

$$t = \frac{d}{r_y}, \quad x_{\text{cam}} = t \cdot r_x, \quad y_{\text{cam}} = t \cdot r_z$$

5. Construct a rectified top-down camera-frame image where pixel distances correspond to real physical distances (pixels-per-meter scale).

This make sure that QR codes have the ideal geometric properties on which algorithms/models

are built. Low-light conditions necessitate this additional preprocessing since it made significant difference to our pipeline.

B) Detection of QR Candidates

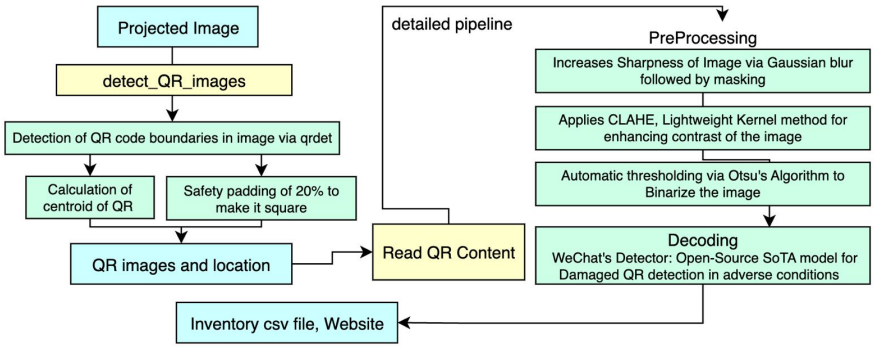


Figure 5.4: QR code processing pipeline from projected image to inventory records.

The rectified frame is passed to the YOLO based *QRDetector* model.

1. Detector outputs bounding boxes (x_1, y_1, x_2, y_2) with confidence filtering and NMS (configurable).
2. Each bounding box is:
   - Clamped to image boundaries
   - Converted to centroid (u_c, v_c) for coordinate mapping
   - Padded based on box size (ratio defined in YAML)
   - Cropped from the projection image
3. Cropped regions are converted to a square shape using *_make_square_canvas()* to preserve QR finder pattern ratios.

This produces high-quality QR tiles for decoding.

C) Enhancement Pipeline

Each cropped QR region is normalized via the following transformation stack:

1. Resize to a fixed 400×400 resolution (Lanczos)
2. Apply CLAHE for contrast enhancement
3. Apply Unsharp Mask to sharpen edges
4. Apply binary thresholding for readability

This step significantly improves decoding success in low-light warehouse environments.

D) Decoding (WeChat QR Engine)

Decoding uses OpenCV's WeChat QR inference engine:

$$\text{payload} = \textit{WeChatQRCode.detectAndDecode}(I_{binary})$$

The detector is lazy-loaded to reduce memory overhead and avoid startup delays. Empty or unreadable QRs return a null string.

E) Coordinate Mapping to Global Frame

For each decoded QR:

$$x_{\text{cam}} = \frac{u_c}{\text{ppm}} + x_{\text{cam_origin}}, \qquad y_{\text{cam}} = y_{\text{cam_origin}} - \frac{v_c}{\text{ppm}}$$

Global coordinates incorporate SLAM offsets:

$$x_{\text{global}} = x_{\text{cam}} + x_0, \qquad y_{\text{global}} = y_{\text{cam}} + y_0$$

This allows each QR code to be placed accurately on the warehouse map.

5.2.7 Processing and Data Logging Pipeline

Once QR data and coordinates are available, the processor executes a complete inventory management workflow:

1. **Parse Payload** Format: *R03_S4_ITM240* Extracted as: rack, shelf, item.
2. **Duplicate Checking** Each entry is checked against:
   - Previously scanned items from the same session
   - Distance-based duplicate thresholds on global coordinates of centroid(X/Y meters)
3. **Inventory Update** Depending on the condition:
   - New item $\rightarrow$ create new row
   - Same item, same location $\rightarrow$ ignore
   - Same item, new location $\rightarrow$ create new row
4. **Optional Annotated Image Saving** Boxes are drawn on the full frame and written to the annotated output directory for debugging and visualization purposes.
5. **Remote API POST** Each valid entry is sent to the web server:[*/api/update*]

This completes the closed-loop scanning and inventory registration cycle.

5.3 HMI Design

The human machine interface provides a simple web based view of the rover and its interaction with the warehouse. It is designed to run on the Host PC itself or on a nearby support machine that can reach the robot over the network.

5.3.1 Front-End Structure

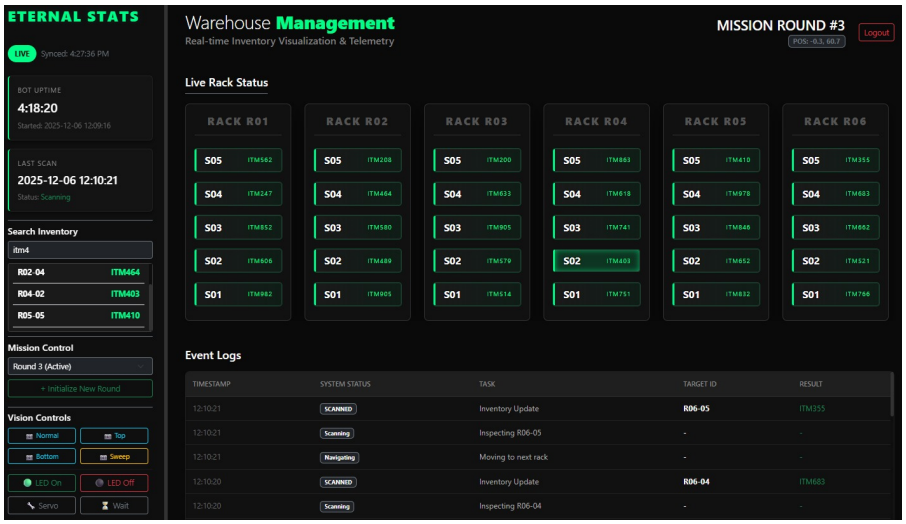


Figure 5.5: Real-time warehouse inventory dashboard showing rack status, search, and event logs

The front end resides in a template directory and consists mainly of two pages:

- ***login.html***: Presents a username and password form. On submission the credentials are sent as a POST request to the Flask back end. Valid credentials create a session and redirect the user to the dashboard. Invalid credentials keep the user on the login page with a short error message.
- ***dashboard.html***: Acts as the main monitoring screen. It displays a vertically scrollable list of rack cards, where each card contains the rack identifier, shelf identifier and Item Code label. Additional widgets show the current run identifier and timestamps of the last updates. The layout is optimised for continuous viewing on a laptop or a control tablet.[1]

The dashboard obtains its data through JavaScript calls to the Flask API. It polls the back end at a fixed rate to fetch the latest JSON snapshots. Small updates, such as a single rack changing from unknown to full, only update the corresponding card on screen.

5.3.2 Back-End Architecture

The HMI back end is a Flask application located in the *backend* folder of the codebase. The main elements are:

- ***app.py***: Entry point that creates the Flask app, configures session handling and registers all routes. It also contains helper functions to read and write CSV files, and to package the data as JSON responses for the dashboard.
- ***data/* directory:** Contains *rover_data.csv*, *racks.csv* and *inventory.csv*. These files store telemetry, rack structure and inventory events. The back end appends new rows for each scan and reads the full set when serving dashboard requests.
- **Utilities:** Small route files and helper scripts such as *restart_run.py* provide endpoints for

[1] Front end behaviour follows the website documentation.

restarting a run or performing local tests.[2]

The overall data path is:

1. Navigation and perception nodes publish scan results and inventory updates.
2. Logging utilities write these events into the CSV files in the *data/* folder.
3. The Flask app parses the CSV files on request and returns JSON objects that group entries by rack and shelf.
4. The dashboard receives the JSON payload and updates the rack cards on screen.

This design keeps the HMI independent of ROS message formats while still reflecting the live state of the robot.

(Space for a sequence diagram that shows ROS topics, CSV logging, Flask routes and dashboard updates.)

5.3.3 Role in the Overall System

The HMI sits at the outer edge of the software stack. It does not send direct motion commands to the robot. Instead, it reports what the robot has already done and what it has observed. Operators use it to:

- confirm that scans have been completed for a set of racks,
- check that the battery is within the safe range before starting long runs, and
- export inventory CSV files for further analysis or import into a warehouse management system.

By keeping the HMI loosely coupled in this way, the navigation and scanning pipelines can evolve without forcing large changes in the user facing components.

5.4 Communication Protocols

5.4.1 Software Stack Overview

The mapping and navigation stack is built on ROS 2 with the Nav2 framework acting as the main navigation layer. On top of the base ROS 2 middleware, we use the following core packages:

- ***slam_toolbox*** for online SLAM and map management.
- ***robot_localization*** for state estimation using an extended Kalman filter.
- ***nav2_bringup*** and the broader Nav2 stack for path planning, control, and recovery behaviors.

The Jetson Orin Nano runs these nodes, while Arduino-based controllers provide low-level sensor and odometry topics over serial bridges. The TF tree follows the standard ROS 2 convention and includes *map*, *odom* and *base_link* frames, with *slam_toolbox* maintaining the *map*→*odom* transform and *robot_localization* maintaining the *odom*→*base_link* transform.

[2]Folder layout based on the website documentation.

5.4.2 Module-wise Architecture

Perception Pipeline

For mapping and navigation, we use three primary sensing modalities:

- a RP 2D LiDAR publishing range scans on */scan* topic,
- an IMU(BNO055) publishing linear acceleration and angular velocity on */imu* topic
- wheel and lift motor encoders combined into odometry messages on */odom* topic.

The *robot_localization* package fuses */imu* and */odom* using an extended Kalman filter configured for a differential-drive platform. This filter outputs a continuous estimate of the robot pose and velocity in the *odom* frame, which is then consumed by both *slam_toolbox* and the Nav2 local planner.

LiDAR data from */scan* is used in two ways:

- by *slam_toolbox* to align successive scans and refine the *map* and *map*→*odom* transform
- by the Nav2 costmap layers to build obstacle and inflation maps for collision avoidance.

The QR-based vision pipeline described in the previous chapter runs in parallel and is not part of the navigation loop. Its role is to identify rack locations once the mapping and navigation stack has brought the robot to the desired scan poses.

Chapter 6

Integration & Control

6.1 Hardware & Software Interactions

The motion and control architecture is organised in three main layers:

- **High-level compute layer (Jetson Orin):** Runs localization, mapping, vision pipeline, path planning, and mission logic and decides where the robot should go and what rack levels should be scanned.
- **Low-level motion control layer (Arduino Mega 2560):** Receives velocity commands and target Z positions from the Jetson Orin over UART (via a USB-to-UART interface). It generates PWM and direction signals for the X–Y drive motors and the Z-axis motor, and it closes the control loops using encoder feedback.
- **Camera orientation layer (ESP32 Servo Driver + Servo):** Orientation control is handled by the ESP Bus Servo Driver, which accepts simple angle commands and actuates the camera servo to achieve the desired orientation. This subsystem operates independently of the differential drive motion and the Z-axis lift assembly.

The Jetson Orin computes the desired linear and angular velocities for the base and the target positions for the Z-axis, as well as any camera orientation updates. These commands are sent to the Arduino Mega and the ESP32 Servo Controller over their respective communication links. The microcontrollers then handle the timing-critical tasks of generating PWM signals, reading encoders, and driving the servo. This layered design follows a central–peripheral architecture, in which a single high-level compute unit issues abstract motion and perception directives while multiple dedicated peripheral controllers execute precise, time-sensitive actuation. This separation ensures deterministic motor control, reduces computational load on the central processor, and preserves system robustness under real-time operating constraints.

6.2 Control Flow

The mapping and navigation stack is built on ROS 2 with the Nav2 framework acting as the main navigation layer. On top of the base ROS 2 middleware, the following core packages are used:

- ***slam_toolbox*** for online SLAM and map management.
- ***robot_localization*** for state estimation using an extended Kalman filter.
- ***nav2_bringup*** and the broader Nav2 stack for path planning, control and recovery behaviours.

The Jetson Orin Nano runs these nodes, while the Arduino-based controllers provide the low-level sensor and odometry topics over serial bridges. The TF tree follows the standard ROS 2 convention and includes *map*, *odom* and *base_link* frames, with *slam_toolbox* maintaining the *map*→*odom* transform and *robot_localization* maintaining the *odom*→*base_link* transform.

For mapping and navigation three primary sensing modalities are used:

- a 2D RPLiDAR publishing range scans on the */scan* topic,
- an IMU (BNO055) publishing linear acceleration and angular velocity on the */imu* topic,

- wheel and lift motor encoders combined into odometry messages on the */odom* topic.

The *robot_localization* package uses an extended Kalman filter configured for a differential-drive platform. This filter outputs a continuous estimate of the robot pose and velocity in the *odom* frame, which is then consumed by both *slam_toolbox* and the Nav2 local planner.

LiDAR data from */scan* is used in two ways:

- by *slam_toolbox* to align successive scans and refine the *map* and *map*→*odom* transform
- by the Nav2 costmap layers to build obstacle and inflation maps for collision avoidance.

Simultaneous localisation and mapping is handled by *slam_toolbox* running in online mode. The node subscribes to */scan* and the fused TF from *robot_localization*, incrementally building a 2D occupancy map of the warehouse racks and aisles. Loop closures are enabled so that drift is corrected when the robot revisits known areas.

On top of this, the Nav2 stack provides global and local planning:

- The **global planner** operates in the *map* frame and computes paths around racks and static obstacles.
- The **local planner** operates in the *odom* frame, using LiDAR-based costmaps to track the global path while reacting to dynamic obstacles.

For autonomous rack scanning, the *nav2_simple_commander* Python API is used. A mission script instantiates a commander node, specifies a sequence of poses in front of the target racks, and calls *go_to_pose()* for each waypoint. Completion and error states are monitored at the ROS 2 level, and the mission can be paused, resumed or aborted from the same interface.

Overall, the navigation pipeline can be viewed as:

$$\begin{aligned} &\text{/scan, /imu, /odom} \rightarrow \text{robot_localization} \\ &\rightarrow \text{slam_toolbox} + \text{Nav2 costmaps} \\ &\rightarrow \text{Nav2 planner} + \text{controller} \rightarrow \text{motor commands.} \end{aligned}$$

6.3 Fail-safe Mechanisms

Our robot includes several fail-safe features so that unexpected events do not damage the hardware or corrupt the navigation state. These mechanisms are simple to operate and are placed so that a warehouse operator can intervene quickly if required.

- **Limit switches on the Z axis:** Mechanical limit switches are mounted at the top and bottom of the vertical mechanism. If the carriage ever travels beyond its intended range, the switches cut motion commands to the Z motor and prevent further travel, protecting the belt, guides and motor from overload.
- **Emergency stop switches on both sides:** An emergency stop push button is provided on each side of the robot. Either switch can be pressed to disconnect power to all actuators, which brings the base and the lift to a controlled halt. The placement ensures that at least one switch is reachable regardless of the operator's position around the robot.
- **Preservation of navigation state:** Pressing the emergency stop does not power off the Jetson. The SLAM, map data and localisation state remain active in memory. Once the emergency stop is released and the system is re-enabled, the robot can resume its mission from the same global frame without repeating the full mapping process.

Chapter 7

Testing & Validation Summary

This chapter summarises how the main subsystems of the rover were tested in practice: odometry, Z motion feedback, and the camera plus vision pipeline.

7.1 Odometry Validation

The goal of this test was to check whether the reported pose of the robot matches the actual motion on the floor.

- A rectangular path was marked on the warehouse floor using tape. The dimensions of the path were measured with a measuring tape.
- The robot was driven along this path using teleoperation. During the run, odometry messages were logged at regular time stamps, including x, y and heading angle.
- At key corners of the path, the physical position of the base was recorded and compared with the logged odometry values.
- Heading angles were compared with both the IMU yaw reading and a manual estimate taken from floor markings.

Across repeated trials, the error in x and y stayed within the expected limits for a differential drive platform on a flat surface, and the yaw drift matched the behaviour predicted by the IMU and wheel encoder noise. These results confirmed that the odometry stack is suitable as an input to the EKF and SLAM modules.

7.2 Z Motion Feedback

This test verified the mapping between encoder ticks on the Z axis motor and the actual height of the camera carriage.

- The carriage was moved from the home position to several fixed heights that correspond to shelf levels.
- At each height, the encoder count was recorded together with the measured distance from the base to the camera using a scale.
- The procedure was repeated for multiple up and down cycles in order to capture any slack or backlash in the mechanism.
- Encoder counts were then plotted against measured height and a best fit mapping was derived.

The resulting curve was close to linear across the operating range, and the residual error at each shelf height remained within a few millimetres. This mapping is now used in the software to command the carriage to specific heights and to report camera height for logging.

7.3 Camera and Vision Pipeline

The final set of tests focused on the end to end QR scanning pipeline, from image capture to website display.

Image Capture and Selection

- The robot was parked in front of a rack at a fixed distance. Tests were run with the IMX258 camera and with a higher field of view Lenovo webcam.
- The Z mechanism moved in 20 cm steps and stabilised at each shelf level. At every stop, a burst of 10 images was captured.
- For each burst, the sharpest frame was selected using a Laplacian based focus score. This frame was used for further processing in order to reduce the effect of motion blur or small vibrations.

Rectification and QR Processing

- If the camera was tilted, the selected frame was rectified using the calibrated homography so that the rack face appeared front facing.
- The rectified image was passed to the QR detector model, which produced bounding boxes around QR candidates.
- Each crop was padded, made square and enhanced using CLAHE and unsharp masking, followed by binary thresholding.
- The processed tiles were then decoded using the WeChat QR engine, which produced the final payload strings.

De-duplication and Logging

- For every decoded QR, the system stored payload, camera frame coordinates and global coordinates.
- A de-duplication step compared new detections with existing entries based on payload and location thresholds. Repeated detections of the same code at the same physical position were filtered out.
- Valid entries were written to the inventory CSV and pushed to the website back end.

During these tests, operators monitored three feedback channels: the live web dashboard, log files, and annotated debug images that show bounding boxes and decoded values. This combination allowed quick identification of failure cases and confirmed that the final pipeline reliably converts rack images into structured inventory records.

Chapter 8

Innovation & Novel Contributions

8.1 Design Choices

8.1.1 Software Architecture

Our QR based inventory pipeline is built around a single *Processor* class. The class exposes a small set of high level entry points (*process_image_to_qr_data*, *process_scan_to_inventory*, *process_image*) and organises the work internally into clear modules:

- **File utilities:** Small helper functions create all log, capture and inventory paths before any write, so long autonomous runs do not fail because a folder is missing.
- **Inventory management:** Parsing, duplicate checking and CSV handling live in one place, so the perception stack only hands over payload strings and global coordinates.
- **Coordinate geometry:** All camera and world frame maths like K matrix handling, rotation matrices, plane intersection and UV transforms are grouped into a dedicated set of methods.
- **Vision and decoding:** Detection, cropping, enhancement and decoding of QR regions are kept separate from inventory logic, which makes the pipeline easy to change or extend.

From the rest of the robot the interface stays simple. The code is asked to take an image, a pose and a scan ID, and it either updates the inventory or reports that no valid code was found.

8.1.2 Geometric Projection before further Processing

Problem 1: QR detection in images suffers from inaccuracy due to geometrical distortions characteristic of the camera itself
Problem 2: Ignoring edges of the image to avoid such distortions, necessitates high overlap between images, which leads to duplication that is hard to resolve

To solve both problems, We project each input image onto a plane parallel to the rack with each QR coincident to its actual position in the global coordinate frame by accounting for both camera orientation and well-calibrated intrinsic parameters (K,D) of the camera itself.

We establish a mapping from pixel centroids (u, v) to the global warehouse frame (x_g, y_g) using the ray-plane intersection model:

$$(x_g, y_g) = f\left(K, D, R_{\theta,\phi}, d, (u, v), \mathbf{t}_{slam}\right)$$

- A calibrated intrinsic matrix K and distortion coefficients D (calibrated to subpixel accuracy) models the intrinsic properties of the camera precisely, more than any AI/ML or otherwise trained algorithm without access to intrinsic parameters of the camera can achieve.
- Rotation matrix $R(\theta, \phi)$ models orientation of camera. Combined with K, D this produces a pixel ray in 3 dimensional space originating at center of the camera.

- Each pixel ray intersected with the rack plane (at distance d), gives the camera frame coordinates $(x_{\text{cam}}, y_{\text{cam}})$, with interpolation to a uniform pixels per meter value.
- A vectorised implementation projects the full image in one pass with low latency.

8.1.3 Duplicate Suppression in Physical Space

We implement a multistep process to check against duplicacy of captured QR before logging it into our database.

- Each decoded QR is assigned a global coordinate $(x_{\text{global}}, y_{\text{global}})$ by vectorially adding SLAM offsets to the camera frame estimate $(x_{\text{cam}}, y_{\text{cam}})$
- We first check if Payload parameters Rack_ID, Shelf_ID, Session_ID match. If so, we compare their physical distance via a normalised distance metric:

$$d_{\text{norm}} = \sqrt{\left(\frac{\Delta x}{\text{threshold}_x}\right)^2 + \left(\frac{\Delta y}{\text{threshold}_y}\right)^2}$$

- If $d_{\text{norm}} \leq 1$, the scan is treated as a duplicate and ignored. If it is larger than 1, a new location is added for the same item.
- Presence of tunable parameters threshold_x and threshold_y ensure we can accomodate differential errors in different axes as per the formula

$$\text{Threshold} = \min(\text{qr_side length}, \sigma_{\text{motion}})$$

Working in physical space makes the duplicate logic tolerant to small pose errors while still separating truly different rack positions.

8.1.4 Asynchronous Orchestration and Fault Tolerance

Problem 1: Sequential execution of vision tasks causes blocking delays, where the robot must halt navigation while waiting for computationally expensive QR decoding and model enhancement.
Problem 2: Monolithic perception scripts lack state safety and granular error handling, making them brittle and difficult to debug during autonomous operation.

To resolve these integration bottlenecks, we re-architected the solution into a decoupled Producer-Consumer model wrapped in a Singleton orchestrator. This design isolates high-frequency hardware triggers from high-latency image processing, ensuring the main ROS loop remains non-blocking while maintaining production-grade reliability.

- **Multiprocessing Architecture:** We utilize a *multiprocessing.Queue* to decouple the *Scanner* (Producer) from the *Processor* (Consumer). The *Scanner* operates on the main thread for fast burst capture, while a separate daemon process handles the heavy CPU load of rectification and WeChat inference.
- **Singleton Orchestration:** The *visionNode* class implements a Singleton pattern (via *visionNode.get()*), serving as the single source of truth for hardware state. This prevents race conditions when accessing shared resources like the camera or servo controller from different

ROS callbacks.

- **Graceful Degradation & Traceability:** The entire pipeline is wrapped in comprehensive *try/catch* blocks to prevent abrupt crashes during hardware timeouts. A unified logging system records events with severity levels (*INFO*, *WARN*, *ERROR*) to timestamped files, creating a persistent audit trail for debugging.
- **Production-Grade Packaging:** The codebase is structured as a modular Python package rather than a collection of loose scripts. This allows the entire vision stack to be installed and imported directly into the ROS workspace as a native dependency, standardizing the interface between the perception layer and the navigation stack.

8.2 Algorithm Optimisations

8.2.1 Contrast Aware QR Preprocessing

The QR decoding path uses a short image processing chain:

- QR regions are padded and placed into a square canvas, so that the finder patterns are not stretched.
- CLAHE is applied to handle strong shadows and local contrast changes that appear on different types of packaging.
- An unsharp mask strengthens edges before binary thresholding, which creates clean module transitions for the decoder.

This pipeline gives stable reads on small ($5 \times 5\,\text{cm}$) codes at typical rack distances and does not depend on heavy neural models.

8.2.2 Detector and Decoder Pairing

Detection and decoding are separated in the implementation:

- A neural QR detector (*QRDetector*) is used only to find bounding boxes in the rectified image.
- A WeChat based QR decoder is loaded only when a region needs to be read, which keeps memory usage low during long runs.

This pairing lets us tune detection thresholds and non maximum suppression settings without touching the decoding backend and makes the system easier to adapt across camera setups.

8.3 Cost and Efficiency Innovations

- **Single pass rectification:** The projection from image to wall plane is written as a vectorised operation, so there are no per detection warps and CPU load stays predictable.
- **Configurable thresholds:** All geometry and duplicate thresholds are passed as constructor parameters. The same code can be used in warehouses with different rack spacing without retraining.
- **Lean logging and CSV storage:** Inventory is stored in a CSV file with simple fields for rack, shelf, item and coordinates, which keeps post processing and integration work small.

Chapter 9

Limitations & Future Scope

9.1 Known Limitations

- **Differential drive:** The base is differential drive, so the robot cannot move directly sideways. Lateral repositioning always needs a small rotate–then–drive motion. This is slower than a holonomic base but keeps odometry and the SLAM model simple and reliable.
- **Lighting conditions:** The vision pipeline is tuned for low and medium light dark store conditions. Under very bright light or strong glare, the current camera settings are not fully optimised and may need site specific exposure and gain tuning.
- **QR and environment quality:** The system assumes clean, high contrast QR labels mounted flat on the rack face and a relatively stable layout. Damaged labels or large layout changes (moved racks, permanently blocked aisles) reduce scan and navigation quality and require relabelling or a short remapping session.

9.2 Commercialization Path

- **Custom integration PCB:** The current multi–Arduino wiring is replaced by a single custom PCB that combines controller, sensor inputs and power rails. This reduces loose wiring, speeds up assembly and improves electrical reliability for field units.
- **Serviceable modules:** Wear parts such as the battery, camera module and vertical drive belt remain accessible and can be swapped without dismantling the chassis. The PCB uses standard connectors so that on–site service is based on module replacement rather than board repair.
- **Configurable deployment:** Camera intrinsics, rack spacing, QR sizes and motion limits are stored in configuration files. This allows quick commissioning of new customer sites without changes to the core codebase.

9.3 Scalability

- **Higher resolution camera:** Using a higher resolution camera increases the effective field of view at each stop, so more shelves can be covered per height. This cuts scan time per rack and makes the same platform suitable for larger warehouses.
- **Multi robot operation:** The navigation and inventory logic are implemented as ROS 2 nodes with clear topic interfaces. The same stack can be extended to a small fleet where robots share a map and split rack ranges under a higher level coordinator.
- **System integration:** Inventory outputs are already stored in structured CSV form and served by the web back end. These records can be connected to existing WMS or ERP systems, so that scaling to more sites mainly involves data integration, not changes in robot behaviour.

Chapter 10

Innovation Highlights

10.1 Innovative Approaches

- **Image-to-inventory pipeline:** The *process_image* method implements an end–to–end flow from a single camera frame to a validated inventory update, including geometry, decoding, duplicate suppression and logging. Higher–level software only needs to provide the image, pose offsets and a scan identifier.
- **SLAM–aligned QR mapping:** By anchoring each decoded QR in the same global frame used by the SLAM stack, the system turns visual detections into structured rack observations that can be overlaid directly on the warehouse map.
- **Fail–soft behaviour:** Invalid payloads, missing images and failed detections are handled with explicit status codes (*ERR_BAD_PAYLOAD*, *NO_QR_DETECTED*, etc.), which allows the mission logic to recover gracefully without manual resets.
- **Decoupled Producer–Consumer Architecture:** A multiprocessing design separates high frequency hardware triggers (Scanner) from computationally intensive rectification and decoding (Processor). This ensures the main ROS navigation loop remains non blocking and deterministic, regardless of the visual processing load.
- **Modular Python Packaging:** Unlike typical ad–hoc scripts, the vision stack is encapsulated as a distinct, installable Python package. This ensures clean, version–controlled integration into the ROS workspace as a native dependency.

10.2 Algorithms Used

- **Camera and projection model:** Analytic rotation matrices derived from pitch and yaw angles, combined with a calibrated intrinsic matrix, are used to project pixel rays onto a planar rack face at known distance.
- **QR detection and decoding:** A deep–learning QR detector is used for robust localisation, while the WeChat decoder handles the final bit–level decoding on contrast–enhanced crops.
- **Duplicate checking metric:** A normalised Euclidean metric in metres, with separate thresholds along aisle and rack directions, decides whether a scan corresponds to an existing entry or a genuinely new location for the same item.
- **Entropy–Based Frame Selection:** To counteract motion blur and vibration from the mobile base, the system employs a burst–capture algorithm. It computes Laplacian variance scores (σ^2) in real–time to isolate and persist only the sharpest frame from a capture sequence, significantly improving downstream detection recall.
- **Vectorised Geometric Rectification:** The projection of oblique views is implemented as a single–pass vectorised operation using NumPy. This allows for low–latency rectification of high–resolution images without the performance overhead of iterative pixel–by–pixel loops.

10.3 Status Indicators and Alerts

The rover uses lights, sound and limit sensors to give clear feedback to the operator and to protect the moving parts.

LED indicators

- A small LED panel shows the current state of the robot at a glance.
- Typical patterns include normal operation, warning, and stop or fault.
- Colours follow standard safety practice: red for danger or stop, yellow for warning, blue for required action, and green for safe or ready state.

Buzzer alerts

- An active buzzer provides sound alerts so that the robot state is noticeable even when the LEDs are not in view.
- Different beep patterns indicate startup, proximity alerts and fault conditions.
- If you hear a repeated or unfamiliar pattern, stop the robot and check the dashboard or call support.

Limit sensors

- Limit sensors are fitted on the vertical mechanism to detect end positions of the moving carriage.
- When a limit is reached, the controller stops further motion and may trigger a light or sound alert.
- These sensors protect the lift and camera from hitting mechanical stops and help keep operation safe over long use.

Chapter 11

Testing & Validation

11.1 Z-axis Motion

We first callibrate the Z-axis motion of the camera by plotting distance vs the ticks in relevant encoder. We achieved a consistent and precise result of 163 ticks per centimeter

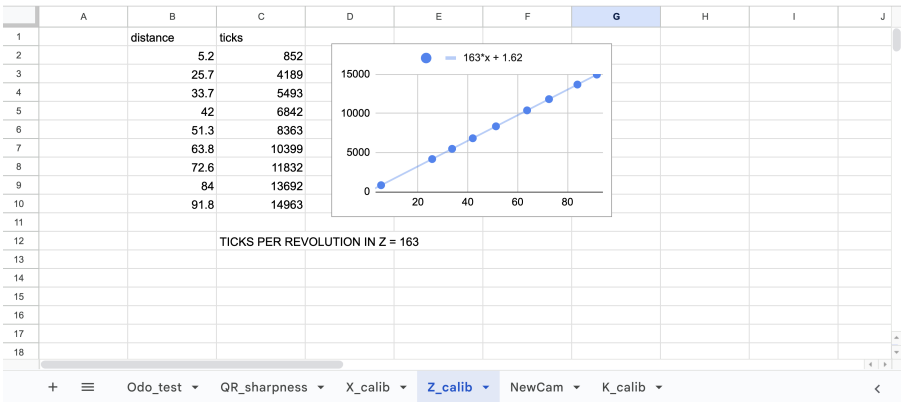


Figure 11.1: QR code processing pipeline from projected image to inventory records.

11.2 Drive Motion

We tested odometry of our drive motion by calibrating distance moved on the X-axis. after many iterations of correcting the drive, we have achieved linear motion as required, with 1523 motor ticks = 1 cm.

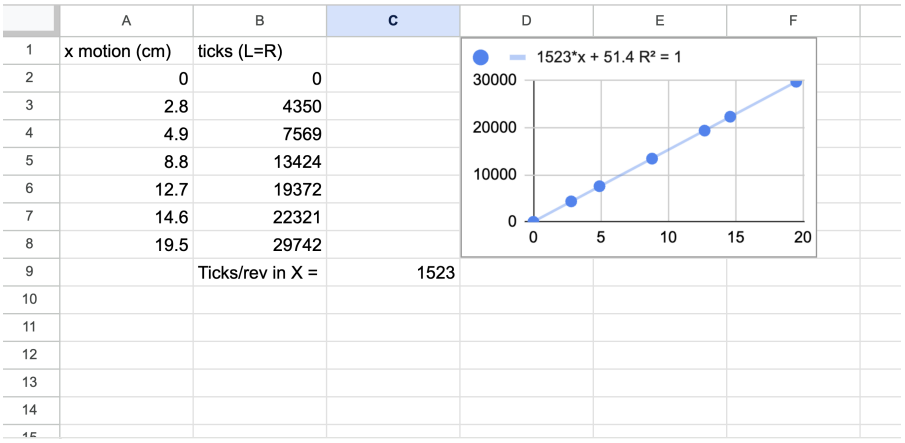


Figure 11.2: QR code processing pipeline from projected image to inventory records.

11.3 Image capture

our robust detection pipeline reliably saves annotated images, logs everything on inventory.csv, and logging duplicated data on website.

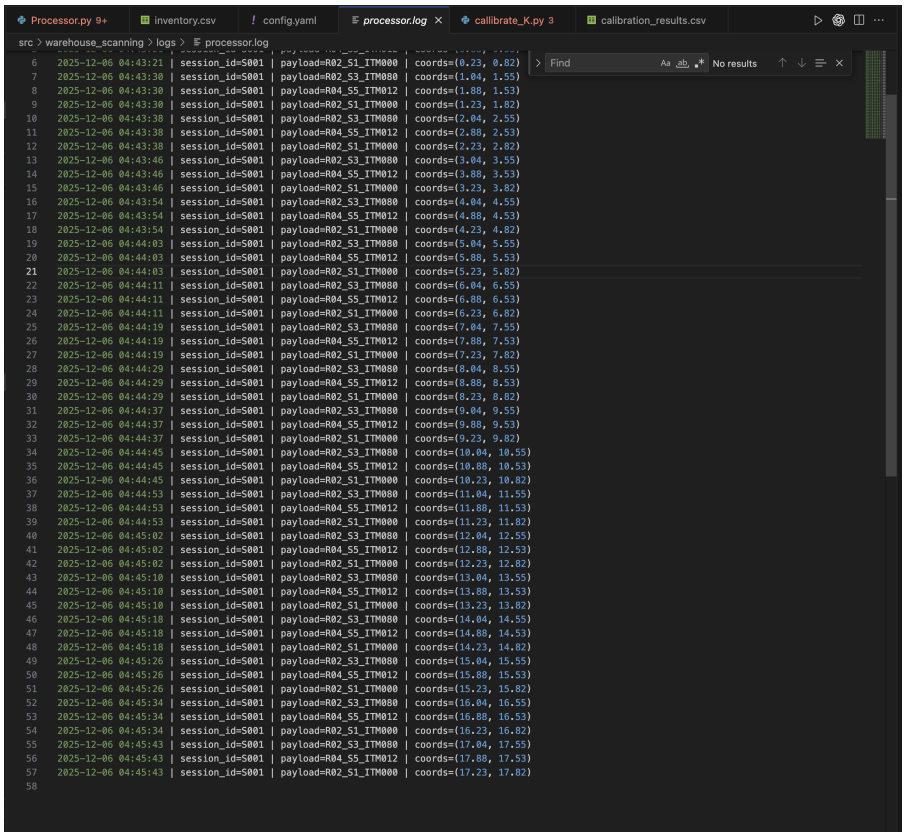


Figure 11.3: Inventory.csv

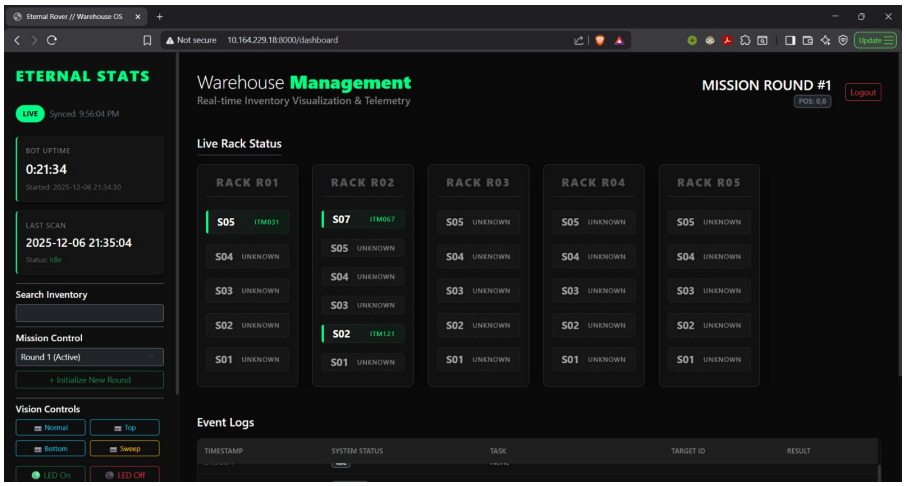


Figure 11.4: Inventory.csv

11.4 Navigation

The given image shows our robus image detection pipeline, showing both the Annotated image saved, and the logs on inventory.csv for the same.

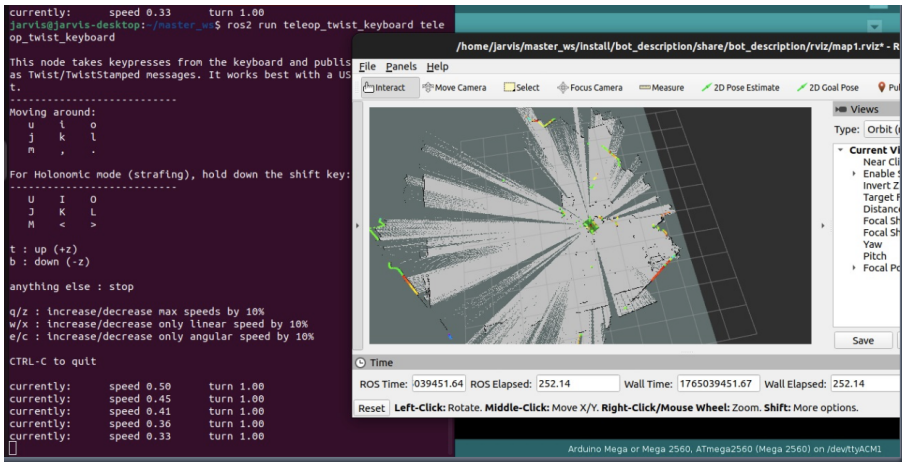


Figure 11.5: Map generation via Nav2 and SLAM

Chapter 12

Maintenance

12.1 Preventive Maintenance

- **Daily:** Wipe the camera lens and LiDAR window with a dry microfiber cloth, check the battery voltage on the display, and visually confirm that all cables and connectors are seated properly.
- **Weekly:** Inspect the Z–axis rails and belt for dust, wear, or slack. Tighten the belt if required and check that all mounting bolts on the vertical rail and base plates are firmly fastened.
- **Bi-monthly:** Apply light lubrication to the Z–axis linear guides and moving pulleys, clean dust from the motor cooling fins and electronics bay, and verify that the encoder connectors are free from play.
- **Yearly:** Replace the rubber grips on the drive wheels if the tread is worn, perform a fresh camera calibration and update the intrinsic parameters in the configuration file, and verify the home positions of the pan and tilt servos by re-running the servo calibration procedure.

12.2 Replacement Parts

The following parts can be replaced without dismantling any major assembly of the robot.

12.2.1 Electrical and Control Units

- **Battery:** The lithium–ion battery used on the robot has an expected life of about 500–1000 charge cycles. After this the capacity drops and it may become unsafe to use. The pack can be replaced by any lithium–ion battery with similar voltage and current ratings. Access is provided by removing the top plate of the base (eight bolts). Suitable packs are available from manufacturers such as Prorange, LRSA, Samsung or as custom packs.
- **Microcontroller:** The Arduino Mega can fail because of overheating or overcurrent and may need periodic replacement. To keep the same wiring and software, it should be replaced with a compatible Arduino Mega 2560. Such boards are easily available from vendors like Robu, ThinkRobotics or Robocraze and can usually be delivered within a few days. Replacement is done by unplugging the DC jacks and gently lifting the board from the PCB header.
- **Servo motors:** The pan and tilt servos that move the camera can be replaced as individual units when wear, backlash or noise increases. The control software maintains a configurable home position for both joints. After a servo change, the technician moves the camera to the desired neutral pose, notes the new angles and updates the home position entries in the configuration file. This keeps all scanning poses consistent over the life of the robot.
- **Camera and calibration:** All camera parameters are stored in a human–readable configuration file *config.yaml*. Code is written in a fail-safe format. This facilitates:
    1. Intrinsic matrix entries, distortion coefficients and focus settings can be edited there directly in case of change over time

2. The software bundle includes a calibration script, an example image folder that shows how to capture calibration views, and a small text guide with command line instructions to disable autofocus and set a fixed focus value. When the camera is replaced, the operator runs the calibration procedure, writes the new parameters into *config.yaml* and the vision pipeline continues to produce accurate projections.
3. In case of change in resolution of camera (new camera installed) we have provided failsafe values for parameters such as interpolation ppm, height and width values for image array etc. based on input image. This ensures smooth continuous functioning of the bot.

12.2.2 Mechanical Structures

- **Base plates:** The CAD files and detailed drawings of the base plates are supplied with this document. Plates can be cut from 3 mm acrylic or aluminium using laser cutting or waterjet machines. Replacement in the field is done by removing the hex–head bolts with standard Allen keys and transferring the mounted components.
- **Camera carriage:** The camera carriage is 3D printed from PLA+ filament. The CAD model is provided and can be printed on common printers such as a Creality Pro in about 3–4 hours. Fitting a new carriage only requires removal of the top pulley on the vertical mechanism, sliding out the old part and mounting the new print.

All other structural and electronic parts have long service lives and high mechanical strength. If a rare failure occurs, most electronic components can be sourced again from Robu or ThinkRobotics. Hardware parts can be fabricated from the supplied CAD models and drawings using standard workshop tools.

12.3 Storage and Transport

The vertical rail of the robot can be removed by loosening four bolts and lifting the assembly off the base. Once the rail is detached, the base fits inside a box of dimensions $600 \times 450 \times 380$ mm for storage and transport. The vertical mechanism can be carried separately, since it is thin and comparatively lightweight. All electronic components are securely fastened to the structure, so they do not move or detach during handling. This makes the robot straightforward to pack, move and reassemble on site.

Chapter 13

References & Appendix

References

- ISO 3691-4:2020 — *Safety of Industrial Trucks: Driverless Trucks*.
- RIA R15.08-1:2020 — *Industrial Mobile Robots — Safety Requirements*.
- ROS 2 Navigation (Nav2) Documentation, https://navigation.ros.org
- REP-105 — *Coordinate Frames for Mobile Robotics*.
- Durrant-Whyte, H., Bailey, T. "Simultaneous Localization and Mapping (SLAM): Part I & II." *IEEE RAM*, 2006.
- Mur-Artal, R. et al., "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras."
- Zhang, Z. "A Flexible New Technique for Camera Calibration." *IEEE PAMI*, 2000.
- Hartley, R., Zisserman, A. *Multiple View Geometry in Computer Vision*, 2nd Ed.
- Brown, D. C. "Decentering Distortion of Lenses."
- Gonzalez, R. C., Woods, R. E. *Digital Image Processing*.
- Wiener, N. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*.
- WeChat QR Code Detector (Tencent AI Lab).
- Smith, A. et al. "Deep Learning Based QR Localization and Super-Resolution."

Appendix

Our system originally used the IMX258 camera; however, it broke during the testing phase. We installed a simple webcam as replacement Due to our modular calibration workflow and geometric projection pipeline, the new camera was fully integrated and calibrated within 30 minutes, without altering any downstream perception code.

The servo tilt mount was designed specifically for IMX258, therefore the webcam operated without tilt degrees of freedom. Despite the reduced DFOV and fixed tilt, the projection and SLAM-aligned mapping pipeline remained fully functional.

Camera Calibration

IMX258 (Old)

Sensor: 1/3.06", Aperture: f/2.2, Focal length: 1.94 mm, FOV: 123° (D), 100° (H), 83° (V).

$$K = \begin{bmatrix} 252.48 & 0 & 314.24 \\ 0 & 254.08 & 231.92 \\ 0 & 0 & 1 \end{bmatrix}, \quad d_{\text{dist}} = [-0.0528,\ 0.0717,\ 0.00231,\ -0.00724,\ -0.03682]$$

Lenovo Webcam (New)

DFOV:95° RMS error $= 1.77$, reprojection mean $= 0.248$ px.

$$K = \begin{bmatrix} 1030.49 & 0 & 978.10 \\ 0 & 1051.62 & 644.15 \\ 0 & 0 & 1 \end{bmatrix}, \quad d_{\text{dist}} = [0.0649,\ -0.4896,\ 0.0283,\ -0.0068,\ 0.6570].$$

Coordinate Frames

UV Frame: pixel coordinates (u, v) with origin at top-left. **Camera Frame**: formed by undistorted normalized rays using K^{-1} and Brown–Conrady distortion. **Global Frame**: QR locations expressed in SLAM coordinates via:

$$t = \frac{d}{r_y}, \quad (x_{\text{cam}}, y_{\text{cam}}) = t(r_x, r_z), \quad (x_g, y_g) = (x_{\text{cam}} + x_{\text{SLAM}},\ y_{\text{cam}} + y_{\text{SLAM}}).$$

Projection Model Summary

Our QR projection pipeline is fully geometric and camera-agnostic:

1. Undistort pixel (u, v) to obtain a normalized camera ray.
2. Apply rotation $R(\theta, \phi)$ from servo orientation (or fixed mount).
3. Intersect ray with the planar shelf surface at distance d.
4. Transform to global frame using robot SLAM pose.

This model follows standard pinhole geometry and the calibration technique of Zhang, ensuring consistent behavior across different cameras and mounting configurations.